Tugas Besar 1

IF3070 Dasar Inteligensi Artifisial



Dibuat oleh:

G26 Depa Ganteng

Naufalziyadh Alif Bintang S	18224077
Muhammad Afif Habiburrahman	18224099
Raditya Nanda Dhevara	18224111

Dosen Pengampu:

Ir. Windy Gambetta, M.B.A.

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

Link REPO: **3**
BAB I **4**
1.1. Latar Belakang & Tujuan 4
1.2. Persoalan 4
1.3. Ruang Lingkup 5
BAB II **6**
2.1. Komponen Bersama 6
2.1.1. Entities 6
2.1.2. Initial 8
2.1.3. Neighbor 9
2.1.4. Objective 11
2.1.5. Validation 12
2.2. Algoritma 13
2.2.1. Genetic Algorithm 13
2.2.2. Simulated Annealing 24
2.2.3. Hill-Climbing Steepest Ascent 29
2.2.4. Hill-Climbing with Sideways Move 34
2.2.5. Random Restart Hill-Climbing 36
2.2.6. Stochastic Hill-Climbing 38
2.3. Hasil Eksperimen tiap Algoritma 43
2.3.1. Genetic Algorithm 43
2.3.2. Simulated Annealing 62
2.3.3. Hill-Climbing 64
a. Hill-Climbing Steepest Ascent 64
b. Random Restart Hill-Climbing 67

c. Stochastic Hill-Climbing....70
d. Hill-Climbing with Sideways Moves....73
BAB III....75
3.1. Kesimpulan....76
3.2. Keterbatasan....76
3.3. Saran....77
BAB IV....77
BAB V....79

Link REPO:

https://github.com/naut1ka/IF3070_G26_Depa-Ganteng.git

BAB I

DESKRIPSI PERSOALAN

1.1. Latar Belakang & Tujuan

Suatu perusahaan mengoperasikan kapal untuk mengangkut berbagai kendaraan dengan dimensi, berat, tarif pengiriman, dan deadline yang berbeda-beda. Perusahaan perlu menentukan kendaraan mana saja yang akan dimuat dan bagaimana posisi kendaraan tersebut di dalam kapal. Namun, proses pemilihan dan penempatan kendaraan yang masih dilakukan secara manual dapat menyebabkan ruang geladak kapal tidak dimanfaatkan dengan baik. Selain itu, kombinasi kendaraan yang dipilih belum tentu menghasilkan total tarif pengiriman yang paling menguntungkan bagi perusahaan.

Oleh karena itu, kami mencoba menggunakan beberapa algoritma untuk membantu menentukan kendaraan yang sebaiknya dimuat dan bagaimana posisi kendaraan tersebut di dalam kapal. Tujuannya adalah mencari susunan kendaraan yang dapat memanfaatkan ruang kapal dengan baik tanpa membuat kendaraan saling bertumpuk atau melebihi kapasitas berat kapal. Kami juga ingin membandingkan hasil dari setiap algoritma untuk melihat algoritma mana yang menghasilkan total tarif pengiriman paling tinggi.

1.2. Persoalan

Permasalahan yang dibahas adalah terkait menentukan kendaraan yang akan dimuat ke kapal dilihat dari posisi dan orientasinya. Setiap kendaraan memiliki ukuran, berat, tarif pengiriman, dan deadline yang berbeda. Kapal memiliki ukuran geladak dan kapasitas berat maksimum yang membatasi jumlah kendaraan yang diangkut.

Dalam proses penempatannya, ada beberapa aturan yang harus diperhatikan. Kendaraan tidak boleh saling bertumpuk sehingga setiap sel pada geladak hanya bisa ditempati oleh satu kendaraan. Selain itu, seluruh bagian kendaraan harus berada di dalam batas geladak kapal dan total berat kendaraan yang dimuat tidak boleh melebihi kapasitas maksimum kapal. Susunan kendaraan akan disusun beda-beda yaitu posisi kendaraan dapat ditukar atau dipindahkan ke koordinat lain, dan orientasinya juga dapat diubah. Namun, setiap perubahan tetap harus memperhatikan aturan penempatan yang sudah ditentukan.

1.3. Ruang Lingkup

Tugas besar ini berfokus pada penentuan kendaraan yang akan dimuat ke kapal dengan mempertimbangkan ukuran geladak dan kapasitas berat kapal. Setiap kendaraan memiliki karakteristik yang berbeda sehingga pemilihan kendaraan perlu dilakukan agar total tarif pengiriman bisa maksimal. Untuk menyelesaikan permasalahan tersebut, diterapkan beberapa algoritma pencarian yang digunakan untuk mencari susunan kendaraan yang sesuai dengan batasan kapal. Data kendaraan dan kapal yang digunakan dalam eksperimen juga menjadi bagian dari pengujian untuk melihat bagaimana algoritma yang diterapkan menyelesaikan permasalahan tersebut.

BAB II
PEMBAHASAN

2.1. Komponen Bersama

2.1.1. Entities

Kode	Penjelasan

`class Dimensions(NamedTuple):`
`  w: int`
`  l: int`
 	
`class Orientation(Enum):`
`  HORIZONTAL = "Horizontal"`
`  VERTICAL = "Vertical"`
 	
`@dataclass(frozen=True)`
`class Placement:`
`  x: int`
`  y: int`
`  orientation: Orientation # x = kolom, y = baris, (x, y) = sudut kiri atas`
 	
`@dataclass(frozen=True)`
`class Vehicle:`
`  id: str`
`  shipping_fee: int`
	Entities mendefinisikan banyak objek yang akan dipakai pada sistem.
	
	1. Dimensions: merupakan lebar dan panjang suatu objek. Dia dipakai untuk mendefinisikan ukuran dari tiap vehicle dan juga ship
	
	2. Orientation: mendefinisikan pilihan apakah vehicle HORIZONTAL ataukah VERTICAL
	
	3. Placement: merupakan objek yang menyimpan koordinat x dan y, dan orientasi dari suatu vehicle yang tersimpan di kapal. ((x, y) adalah sudut kiri atas, x adalah kolom, y adalah baris, w adalah kolom, dan l adalah baris.)
	
	4. Vehicle: merupakan kendaraan yang akan dibawa oleh kapal dan menyimpan seluruh atribut yang dia punya (id, shipping_fee, weight, eta, dimensions). Vehicle juga menyimpan fungsi bawaan effective_size(melihat orientation yang

```
    weight: int
    eta: int
    dimensions: Dimensions # w = kolom, l = baris

    def effective_size(self, orientation):
        if orientation == Orientation.HORIZONTAL:  #
HORIZONTAL = (w, l), VERTICAL = (l, w)
            return (self.dimensions.w, self.dimensions.l)
        return (self.dimensions.l, self.dimensions.w)

    def occupied_cells(self, placement: Placement) ->
list[tuple[int,int]]:
        cols, rows =
self.effective_size(placement.orientation)
        cells = []
        for dx in range(cols):
            for dy in range(rows):
                cells.append((placement.x + dx,
placement.y + dy))
        return cells

@dataclass(frozen=True)
class Ship:
    dimensions: Dimensions # # w = kolom, l = baris
    max_capacity: int

State = dict[str, Optional[Placement]]
```

diberikan sebagai argumen pada vehicle, jika VERTICAL dia akan menukar nilai l dan w) dan juga occupied_cells(menunjukkan cells apa saja yang diokupasi oleh kendaraan pada kapal)

5. Ship: merupakan kapal yang membawa kendaraan. Memiliki atribut dimensions dan juga max_capacity yang menjadi batas maksimum total weight dari total kendaraan yang diangkut.

6. State: merupakan dictionary yang menyimpan semua id kendaraan (yang diangkut ataupun tidak) sebagai key dan Placement menjadi isinya. Placement bisa berisi None jika kendaraan yang dimaksud tidak di atas kapal.

Placement, Vehicle, dan Ship dibuat frozen agar atribut mereka tidak bisa diubah saat menjalankan algoritma sehingga state awal dari kendaraan tetap tercatat.

Orientation disimpan di dalam Placement bukan dalam Vehicle karena orientasi dari sebuah kendaraan di dalam algoritma akan sering diubah.

2.1.2. Initial

Kode	Penjelasan

`def initial_state(vehicles: dict[str, Vehicle], ship: Ship) -> State:`
`    state = {vehicle_id: None for vehicle_id in vehicles}`
`    names = list(vehicles)`
`    random.shuffle(names)`
 	
`    for name in names:`
`        vehicle = vehicles[name]`
 	
`        for _ in range(20):`
`            orientation = random.choice(list(Orientation))`
`            cols, rows = vehicle.effective_size(orientation)`
 	
`            max_x = ship.dimensions.w - cols`
`            max_y = ship.dimensions.l - rows`
`            if max_x < 0 or max_y < 0:`
`                continue`
`            x = random.randint(0, max_x)`
`            y = random.randint(0, max_y)`
 	
`            candidate = dict(state)`
`            candidate[name] = Placement(x, y, orientation)`
	Initial.py merupakan fungsi untuk membuat state awal yang acak dari suatu kapal(kendaraan yang dimuat diatasnya) sebelum dijalankan algoritma.
	
	Dibuat state baru yang diisi vehicle_id yang kosong sebanyak vehicle yang ada pada vehicles. Id kendaraan yang asli ditempatkan pada variable baru yang bernama names yang kemudian diacak urutannya agar semua kendaraan dapat kesempatan untuk naik di kapal.
	
	Untuk setiap nama di dalam names dilakukan iterasi yang berisi:
	
	Dilakukan iterasi sebanyak 20 kali (paling banyak 20 kali, dan berhenti segera setelah ada kandidat yang valid) untuk memilih secara random orientasi dan posisi di atas kapal.
	
	Batas maksimal x dan y (posisi paling kanan dan paling bawah yang masih membuat kendaraannya tetap di dalam kapal) dari sebuah kendaraan berdasar pada panjang dan lebar dari kendaraan.
	
	Dilakukan pengecekan max_x dan max_y

``
            if is_valid(candidate, vehicles, ship):
                state = candidate
                break
 	
    return state
``
	apakah kendaraan dengan orientasi itu muat di kapal. Lalu, menentukan nilai x dan y kendaraan dari range 0 hingga max x dan y secara random.
	
	candidate dibuat untuk menyalin state kendaraan yang sedang berjalan. Lalu entry kendaraan diganti dengan Placement baru berisi atribut x, y, dan orientasi yang sudah dibuat.
	
	candidate divalidasi menggunakan fungsi is_valid, jika diterima maka state diganti dengan candidate dan sebaliknya, jika tidak state tidak akan berubah.
	
	Setelah iterasi selesai, fungsi mengembalikan state.

2.1.3. Neighbor

Kode	Penjelasan

``
def flip(orientation: Orientation) -> Orientation:
    if orientation == Orientation.HORIZONTAL:
        return Orientation.VERTICAL
    return Orientation.HORIZONTAL
 	
def rotate(state: State, vehicle_id: str) -> State:
    new_state = dict(state)
    old = state[vehicle_id]
``
	Neighbor.py berisi fungsi yang akan dilakukan oleh state initial ke state neighbor. Terdapat 4 fungsi yang dipakai yaitu flip, move, rotate, dan swap. Setiap fungsi mengembalikan state baru karena state sebelum diubah tetap harus tercatat. Fungsi-fungsi ini mengubah state yang sedang berjalan (bukan hanya state awal) menjadi tetangganya.

```
    if old is None:
        return dict(state)
    new_state[vehicle_id] = replace (old, orientation =
flip(old.orientation))
    return new_state

def move(state: State, vehicle_id: str, x: int, y: int,
orientation = None) -> State:

    new_state = dict(state)
    old = state[vehicle_id]

    if old is None:
        new_state[vehicle_id] = Placement(x, y,
orientation)

        if new_state[vehicle_id].orientation == None:
            return dict(state)
        return new_state


    if old.x == x and old.y == y:
        return dict(state)

    new_state[vehicle_id] = replace(old, x = x, y = y)
    return new_state

def swap(state: State, id_a: str, id_b: str) -> State:
```

1. flip: fungsi untuk menukar orientasi dari horizontal ke vertical dan sebaliknya.

2. rotate: menggunakan fungsi flip untuk merotate kendaraan dari horizontal ke vertical atau vertical ke horizontal. Jika kendaraan terpilih tidak berada diatas kapal maka langsung dikembalikan state. Hasil dari x dan y tidak berubah, dan hasilnya mungkin keluar kapal atau bertabrakan sehingga harus divalidasi di luar.

3. move: memindahkan kendaraan (termasuk yang diluar kapal) ke atas kapal. Kendaraan yang sudah terpasang dipindah ke (x, y) baru, kendaraan None dengan orientasi dimasukkan ke kapal, dan kendaraan koordinat sama, atau None tanpa orientasi mengembalikan salinan tanpa perubahan.

4. swap: menukar isi entri dua kendaraan (Placement atau None). Kalau salah satunya None, satu kendaraan masuk dan satu keluar, dan kendaraan yang masuk mengambil posisi dan orientasi persis yang ditinggalkan.

``` new_state = dict(state) old_a, old_b = state[id_a], state[id_b] new_state[id_a], new_state[id_b] = old_b, old_a return new_state ```	

2.1.4. Objective

Kode	Penjelasan

``
def value(state: State, vehicles: dict[str, Vehicle]) ->
int:
 	
    total_value = 0
 	
    for vehicle_id, placement in state.items():
        if placement is None:
            continue
        total_value += vehicles[vehicle_id].shipping_fee
 	
    return total_value
``
	Objective function dipakai untuk **menghitung total tarif pengiriman** dari kendaraan masuk ke dalam kapal. Jadi, kita mencari kombinasi kendaraan yang bisa menghasilkan total tarif pengiriman/*shipping fee* setinggi mungkin.
	
	Di kode, total_value diinisialisasi dengan nilai 0, lalu akan ada pemeriksaan setiap kendaraan di dalam state. Kalau placement bernilai None, kendaraan dilewati. Jikalau kendaraan memiliki posisi, nilai shipping_fee-nya ditambahkan ke total_value. Setelah semua kendaraan dicek, return value dari fungsinya adalah total shipping fee tersebut sebagai nilai objective.

2.1.5. Validation

Kode	Penjelasan

`def is_valid(state: State, vehicles: dict[str, Vehicle], ship: Ship) -> bool:`
`    if state.keys() != vehicles.keys():`
`        return False`
 	
`    occupied = set()`
`    total_weight = 0`
 	
`    for vehicle_id, placement in state.items():`
`        if placement is None:`
`            continue`
 	
`        vehicle = vehicles[vehicle_id]`
 	
`        if not isinstance(placement, Placement):`
`            return False`
 	
`        if not isinstance(placement.orientation, Orientation):`
`            return False`
 	
`        cells = vehicle.occupied_cells(placement)`
 	
`        for x, y in cells:`
	State validation digunakan untuk ngecek apakah susunan kendaraan di dalam kapal sudah sesuai dengan batasan yang ada. Jadi, setiap kendaraan yang dimuat harus berada di dalam area kapal, tidak di sel yang sama dengan kendaraan lain, dan total berat muatan tidak melebihi kapasitas maksimum kapal.
	
	Pada awal kode, program melakukan cek apakah state sudah mencakup seluruh ID kendaraan yang ada di vehicles. Untuk occupied itu untuk menyimpan koordinat sel yang sudah terisi, sementara total_weight isinya akumulasi berat kendaraan. Program lalu memeriksa setiap kendaraan yang memiliki posisi. Koordinat dari setiap sel yang ditempati kendaraan didapatkan melalui occupied_cells(), kemudian dicek apakah masih berada dalam batas kapal dan belum digunakan oleh kendaraan lain. Kalau ditemukan koordinat yang keluar dari batas atau bertabrakan, program langsung return False. Setelah semua sel kendaraan diperiksa, beratnya ditambahkan ke total_weight dan dibandingkan dengan kapasitas maksimum kapal. Semisal seluruh pengecekan lolos, fungsi akan return True, yang berarti susunan kendaraan

``
            if x < 0 or x >= ship.dimensions.w:
                return False
 	
            if y < 0 or y >= ship.dimensions.l:
                return False
 	
            if (x, y) in occupied:
                return False
 	
            occupied.add((x, y))
 	
        total_weight += vehicle.weight
 	
    if total_weight > ship.max_capacity:
        return False
 	
    return True
``
	tersebut valid.

2.2. Algoritma

2.2.1. Genetic Algorithm

Genetic Algorithm adalah salah satu algoritma **local search** yang mengambil inspirasi dari cara kerja genetika di dunia nyata. Algoritma ini terdiri atas 3 proses utama, yaitu **parents selection**, **crossover,** dan **mutation.** Implementasi algoritma pada kode akan dijelaskan pada tabel berikut.

Kode	Penjelasan

`@dataclass`
`class GAResult:`
`    initial_state: State`
`    final_state: State`
`    final_value: int`
`    values: list[int]`
`    mean_values: list[float]`
`    iterations: int`
`    duration: float`
`    population_size: int`
	Pendefinisian objek GAResult, yang nantinya menjadi output fungsi genetic_algorithm()
	
	Objek GAResult berisi atribut keluaran sesuai spek yaitu initial_state, final_state, final_value, values, mean_values, iterations, duration dan juga population_size. values menyimpan list value tiap kendaraan, sedangkan mean_values menyimpan mean value kendaraan pada tiap iterasi. Population size akan dijadikan variabel kontrol bersama dengan iterations

`def evaluate_fitness(state: State, vehicles: dict, ship: Ship) -> int:`
`    candidate = state`
`    if is_valid(candidate, vehicles, ship):`
`        return value(candidate, vehicles)`
`    return 0`
	evaluate_fitness()
	
	fungsi ini mengevaluasi nilai fitness state, menggunakan fungsi value() pada objective.py jika state tersebut valid. Jika tidak valid, mengembalikan 0

`def roulette_wheel_selection(population: list[State], fitness_list: list[int]) -> State:`
`    total_fitness = sum(fitness_list)`
`    if total_fitness == 0:`
`        return population[random.randint(0, len(population) - 1)]`
 	
`    pick = random.uniform(0, total_fitness)`
	Parent Selection function #1
	roulette_wheel_selection()
	
	Fungsi ini merupakan salah satu metode pemilihan parent, dimana fungsi akan memilih individu dengan peluang acak yang sebanding dengan nilai kebugarannya (fitness). Individu dengan shipping fee lebih tinggi memiliki porsi "bagian roulette" yang lebih luas, sehingga peluang terpilihnya lebih besar.

``
    current = 0.0
 	
    for i in range (len(population)):
        current = current + fitness_list[i]
        if (current >= pick):
            return population[i]
 	
    return population[-1]
``
	Fungsi akan menghitung nilai total_fitness dari gabungan fitness_value di fitness_list. Kemudian, nilai float acak **pick** dipilih dari range 0 dan total_fitness tersebut. Setelah itu, kita mencari individu dengan nilai pertama yang >= dari pick setelah di loop, yang menandakan current adalah individu dengan nilai fitness dalam float range pick yang terpilih.

``
def sus_selection(population: list[State], fitness_list: list[int], n_parents: int = 2) -> list[State]:
    total_fitness=sum(fitness_list)
    if total_fitness == 0:
        return [population[random.randint(0, len(population))] for _ in range(n_parents)]
 	
    distance = total_fitness/n_parents
    start = random.uniform(0, distance)
    pointers = [start + i * distance for i in range(n_parents)]
 	
    selected_parents = []
    for point in pointers:
        i = 0
        current = fitness_list[0]
        while current < point and i < len(population)-1:
``
	Parent Selection function #2
	sus_selection()
	
	Fungsi ini merupakan metode pemilihan parent menggunakan Stochastic Universal Sampling (SUS), di mana fungsi akan memilih *N* parent sekaligus (n_parents, standar 2) dalam satu kali proses seleksi menggunakan beberapa **pointers** yang memiliki interval jarak yang sama rata.
	
	Fungsi akan menghitung **total_fitness** seluruh populasi. Jika bernilai 0, fungsi mengambil *N* parent secara acak. Jika tidak, fungsi menentukan distance yaitu rentang jarak antar penunjuk ($total_fitness/n_parents$), lalu mengambil nilai acak start antara 0 dan distance. Titik-titik jarum (pointers) dibuat berjejer secara presisi dengan selisih **distance**. Terakhir, fungsi menelusuri akumulasi fitness (current) untuk setiap titik point pada jarum penunjuk, lalu memasukkan individu yang tepat di posisi jarum tersebut ke dalam daftar selected_parents

```
            i+=1
            current+=fitness_list[i]
        selected_parents.append(population[i])

    return selected_parents
```

```
def rank_selection(population: list[State],
fitness_list: list[int]) -> State:

    n = len(population)
    if n == 1:
        return population[0]
    s = 1.5
    order = sorted(range(n), key=lambda i:
fitness_list[i])
    weights = [0.0] * n
    for r, i in enumerate(order): # r ini
rank/position per i
        weights[i] = (2 - s) + 2 * (s - 1) * r / (n -
1)
    return random.choices(population,
weights=weights, k=1)[0]
```

Parent Selection function #3
rank_selection()

Fungsi ini merupakan metode pemilihan parent menggunakan **Linear Rank Selection**, di mana pemilihan tidak didasarkan pada nilai fitness saja, melainkan berdasarkan urutan peringkat (rank) individu di dalam populasi. Metode ini bertujuan untuk menjaga stabilitas tekanan dan mencegah terjadinya konvergensi yang terlalu dini jika ada individu dengan nilai fitness yang melonjak terlalu jauh dibanding individu lainnya.

Fungsi pertama-tama mengambil jumlah populasi (n). Jika populasi hanya berisi 1 individu, fungsi langsung mengembalikan individu tersebut. Selanjutnya, nilai tekanan seleksi ditentukan dengan s = 1.5 (konstanta tekanan, makin tinggi makin menekan) dan indeks populasi terurut dibuat dari fitness terendah hingga tertinggi, di mana r=0 merepresentasikan individu terburuk dan r=n−1 adalah individu terbaik. Fungsi kemudian menghitung **weights** untuk setiap individu berdasarkan peringkatnya. Terakhir, fungsi memanfaatkan random.choices() untuk memilih secara random menggunakan roulette yang

	berbobot dan mengembalikan 1 parent terpilih.

``
def tournament_selection(population: list[State], fitness_list: list[int], k: int) -> State:
    candidate_index = random.sample(range(len(population)),k)
    best_index = candidate_index[0]
 	
    for i in candidate_index:
        if fitness_list[i] > fitness_list[best_index]:
            best_index=i
    return population[best_index]
``
	Parent Selection function #4
	tournament_selection()
	
	Fungsi ini merupakan metode pemilihan parent menggunakan **Tournament Selection**, di mana fungsi memilih **k individu** secara acak dari populasi untuk diadu nilai *fitness*-nya, kemudian mengambil satu individu dengan nilai *fitness* tertinggi di antara peserta turnamen tersebut sebagai *parent*.
	
	Fungsi diawali dengan mengundi **k indeks** kandidat secara acak dan unik dari populasi menggunakan random.sample(). Indeks kandidat pertama dijadikan sebagai pemenang sementara (best_index). Selanjutnya, fungsi melakukan perulangan untuk mengecek nilai *fitness* tiap kandidat terpilih di fitness_list. Apabila ditemukan kandidat yang memiliki nilai *fitness* lebih besar dari kandidat terbaik saat ini, variabel best_index diperbarui. Setelah seluruh k kandidat selesai diadu, fungsi mengembalikan *state* milik individu pemenang turnamen dari population.
	
	4 fungsi ini diuji terlebih dahulu, kemudian saya mengambil fungsi yang mengembalikan average value terbaik, **yaitu tournament_selection()**.

``
def crossover(parent_a: State, parent_b: State, vehicles: dict) -> State:
    vehicles_ids = list(vehicles.keys())
``
	crossover()
	
	Fungsi ini merupakan operasi reproduksi child dengan **Single-Point Crossover**, di mana fungsi

``
    gene_length = len(vehicles_ids)
 	
    if gene_length <= 1:
        return parent_a.copy()
 	
    cutting_point = random.randint(1, gene_length-1)
 	
    child = dict()
    for i in range (0, cutting_point):
        vehicle_id = vehicles_ids[i]
        child[vehicle_id] = parent_a[vehicle_id]
 	
    for i in range (cutting_point, gene_length):
        vehicle_id = vehicles_ids[i]
        child[vehicle_id] = parent_b[vehicle_id]
 	
    return child
``
	mengombinasikan susunan posisi dan orientasi kendaraan dari dua parent_a dan parent_b untuk menghasilkan satu individu child.
	Fungsi diawali dengan mengambil daftar seluruh ID kendaraan (vehicles_ids) dan jumlahnya (gene_length). Jika jumlah kendaraan ≤ 1, pemotongan gen tidak dapat dilakukan sehingga fungsi langsung mengembalikan salinan parent_a. Selanjutnya, ditentukan satu **cutting_point** secara acak di antara indeks 1 hingga gene_length - 1. Segmen state kendaraan sebelum cutting point akan mengambil konfigurasi penempatan dari parent_a, sedangkan segmen kendaraan setelah cutting point diambil dari parent_b. Hasil penggabungan kedua segmen tersebut disimpan di dalam dictionary child dan dikembalikan sebagai state baru.

``
def mutate_candidate(state: State, vehicles: dict, ship: Ship):
    carried = []
    for vehicle_id, placement in state.items():
        if placement is not None:
            carried.append(vehicle_id)
 	
    if carried == []:
        return None
``
	mutate_candidate()
	
	Fungsi ini adalah fungsi mutasi, yaitu fungsi untuk memberikan variasi acak pada individu child untuk menjaga keberagaman genetik.
	
	Fungsi diawali dengan mengumpulkan semua ID kendaraan yang ada di dalam kapal (placement ga None) ke dalam **carried**. Jika kapal kosong, fungsi mengembalikan None. Selanjutnya, jika mutasi dilakukan, fungsi akan memilih satu dari 3 jenis pilihan move yang dapat dilakukan, yaitu **rotate**, **move**, dan **swap**.

```
    kind = random.choice(["rotate","move","swap"])

    if kind == "rotate":
        vehicle_id = random.choice(carried)
        return rotate(state, vehicle_id)

    elif kind == "move":
        vehicle_id = random.choice(carried)
        placement = state[vehicle_id]
        vehicle = vehicles[vehicle_id]
        cols, rows =
vehicle.effective_size(placement.orientation)

        max_x = ship.dimensions.w - cols
        max_y = ship.dimensions.l - rows
        if max_x < 0 or max_y < 0:
            return None

        x = random.randint(0, max_x)
        y = random.randint(0, max_y)
        return move(state, vehicle_id, x, y)

    elif kind == "swap":
        state_a = random.choice(carried)
        others = [n for n in state if n != state_a]
        if others == []:
            return None
```

Untuk penjelasan lebih detail terkait gerakan-gerakan tersebut, bisa dilihat pada penjelasan di bagian ini.

```
        state_b = random.choice(others)
        return swap(state, state_a, state_b)
```

```
def genetic_algorithm(vehicles: dict, ship: Ship,
pop_size: int, max_generations: int, p_mutation:
float, seed: float, elite_size: int = 2, k: int = 3)
-> GAResult:

    random.seed(seed)
    t0 = time.perf_counter()

    population = []
    for i in range(pop_size):
        state = initial_state(vehicles, ship)
        population.append(state)

    fitness_list =
[evaluate_fitness(individual_state, vehicles, ship)
for individual_state in population]
    best_idx = fitness_list.index(max(fitness_list))

    initial_best_state = population[best_idx]
    best_overall_state = population[best_idx]
    best_overall_value = fitness_list[best_idx]

    max_values = []
    mean_values = []
```

genetic_algorithm()

Ini merupakan fungsi utama pada file genetic.py, yang berisi algoritma genetic.

Alur algoritma:

1. Inisialisasi

- Mengatur seed acak
- Membuat variabel t0 untuk memulai penghitung waktu eksekusi
- Membuat population sebanyak pop_size dengan fungsi initial_state()
- Menghitung fitness value semua populasi menggunakan evaluate_fitness(), kemudian mencatat individu terbaik populasi awal

2. Iterasi Generasi

- Pada setiap generasi, fungsi menghitung nilai maksimum dan rata-rata fitness
- Apabila ada individu dengan fitness value yang lebih tinggi dari **best_overall_state**, fungsi akan memperbarui nilai best_overall_state dan best_overall_value
- Elitisme, dimana populasi diurutkan descending, kemudian sebanyak **elite_size** state diambil dan langsung diturunkan ke new_population, untuk menjaga supaya state optimal tidak hilang/teracak

```
    # loop sesuai jumlah iterasi (disini akan disebut
generasi, 1 iterasi = 1 generasi, biar kayak genetic
banget)
    for gen in range (max_generations):
        fitness_list =
[evaluate_fitness(individual_state, vehicles, ship)
for individual_state in population]
        current_max = max(fitness_list)
        current_mean = sum(fitness_list) /
len(fitness_list)

        max_values.append(current_max)
        mean_values.append(current_mean)

        best_idx = fitness_list.index(current_max)

        if current_max > best_overall_value:
            best_overall_value = current_max
            best_overall_state = population[best_idx]

        new_population = []
        sorted_population = [state for _, state in
sorted(zip(fitness_list, population), key=lambda x:
x[0],reverse=True)]

        for z in range (0,elite_size):
```

- Reproduksi dengan seleksi, crossover, dan mutasi (selama size new_population belum mencapai pop_size)
- Seleksi: Memilih parent dengan tournament_selection()
- Crossover: Penggabungan genetik parent dengan crossover()
- Mutation: Jika terpenuhi gachanya, maka mutate_candidate() akan dilakukan. Jika menghasilkan state baru yang valid, child diperbarui
- Memperbarui **population** = **new_population** dan lanjut ke iterasi generasi berikutnya

3. Pengembalian Nilai
- Penghitungan durasi dalam detik
- Pengembalian GAResult yang berisi:
    - initial_best_state
    - best_overall_state
    - best_overall_value
    - histori nilai objective (values dan mean_values)
    - jumlah iterasi
    - durasi eksekusi
    - ukuran populasi.

```
new_population.append(sorted_population[z])

        while len(new_population) < pop_size:
            # Tournament Selection
            parent_a =
tournament_selection(population, fitness_list, k)
            parent_b =
tournament_selection(population, fitness_list, k)

            # Roulette Wheel Selection
            # parent_a =
roulette_wheel_selection(population, fitness_list)
            # parent_b =
roulette_wheel_selection(population, fitness_list)

            # SUS Selection
            # parent_a = sus_selection(population,
fitness_list)[0]
            # parent_b = sus_selection(population,
fitness_list)[1]

            # # Rank Selection
            # parent_a = rank_selection(population,
fitness_list)
            # parent_b = rank_selection(population,
fitness_list)
```

```
            child = crossover(parent_a, parent_b,
vehicles)

            if random.uniform(0,1) < p_mutation:
                mutated = mutate_candidate(child,
vehicles, ship)
                if mutated is not None:
                    child = mutated

            new_population.append(child)

        population = new_population

    duration = time.perf_counter() - t0
    return GAResult(
        initial_state=initial_best_state,
        final_state=best_overall_state,
        final_value=best_overall_value,
        values=max_values,
        mean_values=mean_values,
        iterations=len(max_values),
        duration=duration,
        population_size=pop_size
    )
```

2.2.2. Simulated Annealing

Kode	Penjelasan

`@dataclass`
`class SAResult:`
`  initial_state: State`
`  final_state: State`
`  final_value: int`
`  values: list[int]`
`  probabilities: list[float]`
`  stuck_count: int`
`  iterations: int`
`  duration: float #dalam detik`
	Simulated Annealing merupakan salah satu algoritma AI untuk mencapai keadaan terbaik. Dia mengambil prinsip memandai besi dimana semakin tinggi temperaturnya semakin tinggi kemungkinan untuk berubah bentuk.
	
	simulated_annealing.py berisi pendefinisian objek SAResult, yang nantinya menjadi output fungsi simulated_annealing, fungsi acceptance_probability, random_candidate, dan simulated_annealing.
	
	Objek SAResult berisi atribut keluaran sesuai spek yaitu initial_state, final_state, final_value, values, probabilities, stuck_count, iterations, dan juga duration. values menyimpan list value tiap iterasi dan probabilities menyimpan p tiap iterasi mungkin berpindah atau tidak.

`def acceptance_probability(delta: float, temperature: float) -> float:`
`  if delta >= 0:`
`  return 1.0`
`  return math.exp(delta / temperature)`
	Fungsi ini mengembalikan masukan delta (perbedaan nilai state awal dan state neighbor) dan temperature dimasukkan ke dalam rumus simulated annealing yaitu:
	$p = e^{-\frac{\Delta}{T}}$
	yang nantinya akan digunakan untuk menghitung apakah dia akan pindah ke state neighbor atau tidak.

`def random_candidate(state, vehicles, ship, attempts=20): #note: ini masih belum pasti attempts`	Fungsi random_candidate memilih candidate(state kendaraan lain yang sedang tidak diproses) lallu memilih apakah candidate akan

```
jadi inputan atau udah distate
    carried = []

    for vehicle_id, placement in state.items():

        if placement is not None:
            carried.append(vehicle_id)

    if carried == []:
        return None

    for _ in range(attempts):
        kind = random.choice(["rotate", "move",
"swap"])

        if kind == "rotate":
            candidate = rotate(state,
random.choice(carried))

        if kind == "move":
            vehicle_id = random.choice(list(state))
            placement = state[vehicle_id]

            if placement is None:
                orientation =
random.choice(list(Orientation))
            else:
                orientation = placement.orientation
```

di move, swap, atau rotate.

Variabel carried dibuat untuk menyimpan list kosong. carried akan diisi oleh state dari vehicle_id yang terpilih jika placement tidak None.

Jika carried sudah terisi, iterasi akan dilakukan sebanyak 20 kali untuk menentukan tiap kendaraan yang di dalam carried dilakukan move, swap, atau rotate.

Fungsi yang terpilih lalu akan dijalankan berdasarkan ketentuannya masing masing.

rotate: memilih satu kendaraan secara acak dari daftar carried (kendaraan yang sedang berada di atas kapal), lalu memanggil fungsi rotate() untuk membalik orientasinya dari horizontal ke vertical atau sebaliknya. Posisi x dan y tidak berubah, sehingga hasilnya bisa keluar dari batas kapal atau bertabrakan dengan kendaraan lain. Karena itu, candidate yang dihasilkan tetap harus divalidasi dengan is_valid().

move: memilih satu kendaraan secara acak dari seluruh kendaraan (termasuk yang belum dimuat). Jika kendaraan belum berada di kapal, orientasinya dipilih secara acak, sedangkan jika sudah di kapal, orientasi lamanya dipertahankan. Program menghitung ukuran efektif kendaraan lewat effective_size() untuk mendapatkan max_x dan max_y. Jika kendaraan tidak muat dalam orientasi tersebut, percobaan dibreak. Jika muat, koordinat x dan y dipilih secara acak dalam rentang 0 sampai max_x dan max_y, lalu fungsi

```
            vehicle = vehicles[vehicle_id]
            cols, rows =
vehicle.effective_size(orientation)

            max_x = ship.dimensions.w - cols
            max_y = ship.dimensions.l - rows

            if max_x < 0 or max_y < 0:
                    continue
            x = random.randint(0, max_x)
            y = random.randint(0, max_y)

            candidate = move(state, vehicle_id, x, y,
orientation)

        if kind == "swap":
            a = random.choice(carried)
            others = [n for n in state if n != a]

            if others == []:
                 continue

            b = random.choice(others)

            candidate = swap(state, a, b)

        if is_valid(candidate, vehicles, ship):
```

move() dipanggil untuk memindahkan kendaraan ke posisi baru tersebut.

swap: memilih satu kendaraan a secara acak dari carried (yang berada di atas kapal), lalu memilih kendaraan b secara acak dari kendaraan lain yang bukan a, baik yang berada di kapal maupun tidak. Jika tidak ada kendaraan lain, percobaan dilewati. Lalu, fungsi swap() dipanggil untuk menukar isi Placement keduanya. Jika b belum adalah None, maka a keluar dari kapal dan b masuk dengan posisi dan orientasi persis seperti yang ditinggalkan a.

Setelah candidate terbentuk, dilakukan validasi dengan is_valid(). Jika valid, candidate langsung dikembalikan. Jika tidak valid, iterasi berlanjut ke percobaan berikutnya hingga batas attempts (20 kali). Jika tidak ada candidate valid yang ditemukan, fungsi mengembalikan None. Jika sudah, kembalikan candidate.

``` return candidate return None```	

``def simulated_annealing(vehicles, ship, initial_temp: float, alpha, max_iteration, seed, min_temp=1e-9, stuck_limit = 50) -> SAResult: #note: stuck_limit juga masih belum pasti berapa tergantung besar datanya
 	
    if initial_temp <= 0 or not (0 < alpha < 1):
        raise ValueError("initial temp atau alpha tidak valid")
 	
    random.seed(seed)
 	
    t0 = time.perf_counter()
    state = initial_state(vehicles, ship)
    start = state
    current = value(state, vehicles)
    T = initial_temp
    i = 0
    values = []
    probabilities = []
    streak = 0
    stuck_count = 0
 	
    while T > min_temp and i < max_iteration:
        before = current```
	Fungsi simulated annealing adalah algoritma menentukan state terbaiknya. Berisi parameter: initial_temp: temperatur awal, alpha: faktor pendinginan, max_iteration: batas iterasi, dan seed: mengontrol random melakukan apa. Min_temp: batas temperatur terendah karena jika temp bisa 0, fungsi akan error, stuck_limit (50): jumlah iterasi berturut-turut tanpa perubahan yang dianggap “stuck”. Jika initial_temp <= 0 atau alpha tidak berada di antara 0 dan 1, fungsi mengeluarkan ValueError, karena temperatur harus positif dan terus mengecil.
	
	seed dirandom terlebih dahulu dan disimpan, juga ada t0 yang menyimpan waktu awal algoritma dimulai. Fungsi initial_state dari kapal disimpan di dalam state. current menyimpan value dari state saat ini. T menyimpan temperatur awal dan i untuk penanda iterasinya. values dan probabilities masih menyimpan list kosong. streak untuk menghitung jumlah state jika ia tidak berpindah sehingga jika streak mencapai stuck_limit maka stuck_counter akan bertambah satu.
	
	Iterasi while dimulai dengan syarat T masih diatas min_temp dan juga i < max_iteration. State awal disimpan dulu dalam variable before dan candidate memilih random candidate menggunakan fungsi random_candidate().
	
	Jika candidate tidak None, maka value dari

```
        candidate = random_candidate(state, vehicles,
ship)
        p = 0.0

        if candidate is not None:
            candidate_value = value(candidate,
vehicles)
            delta = candidate_value - current
            p = acceptance_probability(delta, T)

            if p > random.random():
                current = candidate_value
                state = candidate

        values.append(current)
        probabilities.append(p)

        if current != before:
            streak = 0
        if(before == current):
            streak += 1

        if streak == stuck_limit:
            stuck_count += 1

        T = T * alpha
        i += 1
```

candidate akan dicatat dalam candidate_value. Kemudian delta didefinisikan dari candidate value - current.

Dipanggil fungsi acceptance_probability untuk nantinya menentukan apakah state akan pindah atau tidak. Jika p > random.random() maka state akan pindah ke neighbor selama syarat while tadi terpenuhi.

Jika sudah, current dimasukan ke dalam list value dan p juga dimasukan ke dalam probabilities.

Jika, state tidak pindah maka stuck_count akan bertambah satu.

T akan turun setiap iterasinya dengan dikalikan dengan alpha. Iterasi bertambah satu. Setelah while semua, return objek SAResult.

`    return SAResult(start, state, current, values, probabilities, stuck_count, i, duration= time.perf_counter() - t0)`	

2.2.3. Hill-Climbing Steepest Ascent

Kode	Penjelasan

`def find_neighbor(current_state, vehicles, ship, allow_sideways=False):`
`    current_value = value(current_state, vehicles)`
`    best_state = current_state`
`    best_value = current_value`
`    sideways_state = None`
	Bagian ini menginisialisasi variabel yang dipakai sebagai dasar pencarian neighbor.
	- current_value = nilai obj. dari current state
	- Best_state = state dari kandidat terbaik
	- Best_value = nilai shipping fee dari kandidat terbaik
	- Sideways_state = menyimpan kandidat jika sideways_move diizinkan dari parameter allow_sideways (kepake di Sideways biar ngeimplement karena secara garis besar dia sama)

`for v_id in vehicles:`
`        vehicle = vehicles[v_id]`
`        posisi = current_state[v_id]`
 	
`        if posisi is None:`
`            for orient in Orientation:`
`                cols, rows = vehicle.effective_size(orient)`
	Bagian ini untuk mencari kemungkinan placement kendaraan yang belum dimuat dalam kapal. Program ngecek setiap kendaraan dengan posisinya melalui current_state. Kalau ini hasilnya None, maka program akan mencoba setiap orientasi kendaraan dan menghitung ukurannya menggunakan effective_size(). Selanjutnya, while loop x dan y digunakan untuk mencoba koordinat yang mungkin di dalam kapal. Pada setiap percobaan, program membuat salinan current state dan menempatkan

`for y in range(ship.dimensions.l - rows + 1):`
`  for x in range(ship.dimensions.w - cols + 1):`
`  new_state = dict(current_state)`
`  new_state[v_id] = Placement(x, y, orient)`
	kendaraan pada koordinat dengan orientasi yang sedang diuji sebagai kandidat neighbor baru.

`if is_valid(new_state, vehicles, ship):`
`  new_value = value(new_state, vehicles)`
`  if new_value > best_value:`
`  best_state = new_state`
`  best_value = new_value`
`  elif allow_sideways and new_value == current_value:`
`  if new_state != current_state and sideways_state is None:`
`  sideways_state = new_state`
	Bagian ini mengecek apakah kandidat neighbor yang dibuat udah valid atau belum dengan menggunakan is_valid(). Jika valid, program menghitung nilai objectivenya menggunakan value(). Apabila nilai kandidat lebih tinggi dari best_value, kandidat tersebut disimpan sebagai state terbaik sementara. Namun, jika nilai objectivenya sama dengan state saat ini dan allow_sideways bernilai True, kandidat bisa disimpan sebagai sideways_state, selama state-nya berbeda dari state saat ini dan belum ada kandidat sideways yang tersimpan.

``
else:
            for orient in Orientation:
                new_state = dict(current_state)
                new_state[v_id] = Placement(posisi.x, posisi.y, orient)
 	
                if is_valid(new_state, vehicles, ship):
                    new_value = value(new_state, vehicles)
 	
                    if new_value > best_value:
                        best_state = new_state
                        best_value = new_value
 	
                    elif allow_sideways and new_value == current_value:
                        if new_state != current_state and sideways_state is None:
                            sideways_state = new_state
``
	Bagian ini beda dengan bagian sebelumnya yang mencoba memasukkan kendaraan yang belum dimuat ke kapal, bagian ini fokusnya pada kendaraan yang udah memiliki posisi. Program mencoba setiap orientasi kendaraan dengan koordinat x dan y tetap. Setiap perubahan orientasi menghasilkan kandidat neighbor baru lalu diperiksa valid tidaknya dan nilai objectivenya. Jika nilai objective meningkat, kandidat disimpan sebagai best_state. Jika nilainya sama dengan state saat ini dan sideways move diizinkan, kandidat dapat disimpan sebagai sideways_state.

``
for y in range(ship.dimensions.l):
                for x in range(ship.dimensions.w):
                    new_state = move(current_state, v_id, x, y)
 	
                    if is_valid(new_state, vehicles,
``
	Setelah kita mencoba mengubah orientasi kendaraan, program mencoba memindahkan kendaraan yang udah dimuat ke berbagai koordinat di dalam kapal menggunakan fungsi move(). Setiap kandidat perpindahan dicek menggunakan is_valid().

``ship):
                        new_value = value(new_state, vehicles)
 	
                        if new_value > best_value:
                            best_state = new_state
                            best_value = new_value
 	
                        elif allow_sideways and new_value == current_value:
                            if new_state != current_state and sideways_state is None:
                                sideways_state = new_state```	

``vehicle_ids = list(vehicles.keys())
 	
    for i in range(len(vehicle_ids)):
        for j in range(i + 1, len(vehicle_ids)):
            new_state = swap(current_state, vehicle_ids[i], vehicle_ids[j])
 	
            if is_valid(new_state, vehicles, ship):
                new_value = value(new_state, vehicles)
 	
                if new_value > best_value:```
	Bagian ini mencoba mendapatkan hasil kandidat neighbor dengan ditukarnya posisi kendaraan dengan fungsi swap(). Selanjutnya, program mengambil seluruh ID kendaraan, lalu mencoba setiap pasangan kendaraan tanpa mengulang pasangan yang sama dalam urutan terbalik. State hasil pertukaran kemudian dicek.

``
                    best_state = new_state
                    best_value = new_value
 	
                elif allow_sideways and new_value == current_value:
                    if new_state != current_state and sideways_state is None:
                        sideways_state = new_state
``	

``
    if best_value > current_value:
        return best_state, best_value
 	
    if allow_sideways and sideways_state is not None:
        return sideways_state, current_value
 	
    return current_state, current_value
``
	Bagian ini mencari kandidat neighbor dengan menukar posisi kendaraan dengan fungsi swap(). ID semua kendaraan akan diambil, lalu dicoba dengan menukar setiap pasangan kendaraan. Hasil pertukarannya kemudian dicek dan dihitung nilai objectivenya untuk melihat apakah lebih baik dari state saat ini.

``
def steepest_ascent(vehicles, ship):
    current_state = initial_state(vehicles, ship)
    start_state = dict(current_state)
    iterations = 0
 	
    while True:
        best_state, best_value = find_neighbor(
            current_state, vehicles, ship
)
 	
        current_value = value(current_state,
``
	Intinya, bagian ini merupakan core algorithm dari Steepest Ascent. State awal diinisialisasi di initial_state() dan disimpan ke start_state. Lalu, jumlah iterasi diinisialisasi dengan nilai 0, sedangkan history untuk menyimpan nilai objective awal dan setiap kali state berhasil diperbarui. Nextnya, dilakukan pencarian neighbor terbaik menggunakan fungsi find_neighbor() dan membandingkan nilai objectivenya dengan state saat ini. Semisal nilainya lebih baik, state akan diperbarui dan pencarian dilanjutkan. Jika tidak, pencarian dihentikan. Terakhir, ada penghitungan durasi lalu fungsi mengembalikan state awal, state

vehicles)
iterations += 1
 	
if best_value <= current_value:
break
 	
current_state = best_state
 	
final_state = current_state
final_value = value(final_state, vehicles)
 	
return {
"initial_state": start_state,
"final_state": final_state,
"final_value": final_value,
"iterations": iterations
}
	akhir, nilai objective akhir, jumlah iterasi, history nilai objective, dan durasi.

2.2.4. Hill-Climbing with Sideways Move

Kode	Penjelasan
def sideways_move(vehicles, ship): start_time = time.perf_counter() max_sideways = 20	Bagian ini awalnya menentukan batas sideways move sebanyak 20 kali. State awal dibuat dan disimpan ke start_state. Lalu, sideways_moves dan iterations diinisialisasi dengan nilai 0, sedangkan history untuk

`current_state = initial_state(vehicles, ship)`
`start_state = dict(current_state)`
 	
`sideways_moves = 0`
`iterations = 0`
`history = [value(current_state, vehicles)]`
	menyimpan nilai objective dari state awal.

`while True:`
`best_state, best_value = find_neighbor(current_state, vehicles, ship, allow_sideways=True)`
 	
`current_value = value(current_state, vehicles)`
`iterations += 1`
	Ini mencari neighbor terbaik dengan fungsi find_neighbor() lalu mengaktifkan allow_sideways=True sehingga perpindahan ke state dengan nilai yang sama juga diizinkan. Program kemudian menghitung nilai objective state saat ini dan menambah jumlah iterasi.

`if best_value > current_value:`
`current_state = best_state`
`sideways_moves = 0`
	Semisal nilai objective neighbor lebih tinggi, state diperbarui menggunakan best_state dan jumlah sideways move balik ke 0 karena ditemukan peningkatan nilai objective.

`elif best_value == current_value and best_state != current_state:`
`if sideways_moves >= max_sideways:`
`history.append(current_value)`
`break`
 	
`current_state = best_state`
`sideways_moves += 1`
	Kalau nilai objective sama dan state kandidat beda, program akan menjalankan sideways move selama jumlahnya belum mencapai batas, yaitu 20 kali. Setiap perpindahan menambah sideways_moves dan nilai objective state terbaru dicatat ke history. Jika batas tercapai, pencarian dihentikan.

`history.append(value(current_state, vehicles))`	

`else:`
`history.append(current_value)`
`break`
	Ini untuk menghentikan pencarian kalau tidak ditemukan neighbor yang lebih baik atau neighbor dengan nilai sama yang bisa dipilih. Nilai objective terakhir ditambahkan ke history sebelum loop berhenti.

`final_state = current_state`
`final_value = value(final_state, vehicles)`
`duration = time.perf_counter() - start_time`
	Setelah pencarian berhenti, state terakhir disimpan sebagai final_state dan nilai objective akhirnya dihitung. Durasi juga dihitung.

`return{`
`"initial_state": start_state,`
`"final_state": final_state,`
`"final_value": final_value,`
`"iterations": iterations,`
`"sideways_moves": sideways_moves,`
`"history" : history,`
`"duration" : duration`
`}`
	Ini seluruh return value yang akan ditampilkan.

2.2.5. Random Restart Hill-Climbing

Kode	Penjelasan
`def random_restart(vehicles, ship):`	Bagian ini ada penjelasan terkait variabel

``` start_time = time.perf_counter() max_restarts = 10 best_result = None best_value = -1 restart_count = 0 iterations_per_restart = [] history = [] ```	variabel, seperti batas maksimal restart 10 kali percobaan, dan lainnya. Variabel best_result untuk menyimpan hasil terbaik, sedangkan best_value diinisialisasi dengan -1 sebagai nilai pembanding awal agar percobaan pertama nilainya dapat disimpan jika nilai objective-nya lebih tinggi. Variabel restart_count untuk menghitung jumlah percobaan yang dilakukan, iterations_per_restart untuk mencatat jumlah iterasi pada setiap percobaan, dan history untuk menyimpan riwayat nilai objective.

``
    for i in range(max_restarts):
        result = steepest_ascent(vehicles, ship)
 	
        current_value = result["final_value"]
 	
        restart_count += 1
 	
iterations_per_restart.append(result["iterations"])
        history.extend(result["history"])
 	
        if current_value > best_value:
            best_result = result
            best_value = current_value
    duration = time.perf_counter() - start_time
``
	Bagian ini menjelaskan Random Restart dengan menjalankan Steepest Ascent secara berulang hingga batas percobaan. Setiap percobaan dibandingkan berdasarkan nilai objective, lalu hasil dengan nilai tertinggi disimpan. Jumlah iterasi setiap percobaan dicatat menggunakan append(), sedangkan extend() digunakan untuk menggabungkan history nilai objective dari setiap percobaan ke dalam satu daftar.

``
    return{
        "initial_state": best_result["initial_state"],
        "final_state": best_result["final_state"],
``
	Bagian gunanya untuk menampilkan hasil seluruh proses Random Restart. State awal, state akhir, dan nilai objective diambil dari percobaan yang menghasilkan nilai terbaik.

`"final_value": best_result["final_value"],`
`"iterations": best_result["iterations"],`
`"restart_count": restart_count,`
`"iterations_per_restart":`
`iterations_per_restart,`
`"history" : history,`
`"duration" : duration`
	Selain itu, fungsi juga mengembalikan jumlah iterasi pada percobaan terbaik, total restart yang dilakukan, dan jumlah iterasi pada setiap percobaan. Riwayat nilai objective dan durasi total juga ada.

2.2.6. Stochastic Hill-Climbing

Kode	Penjelasan

`def find_better_neighbors(current_state, vehicles, ship):`
`current_value = value(current_state, vehicles)`
`better_neighbors = []`
 	
`for v_id in vehicles:`
`vehicle = vehicles[v_id]`
`posisi = current_state[v_id]`
 	
`if posisi is None:`
`for orient in Orientation:`
`cols, rows = vehicle.effective_size(orient)`
 	
`for y in range(ship.dimensions.l -`
	Melakukan proses pencarian seluruh neighbor yang memiliki nilai objective lebih tinggi daripada state saat ini. Program mencoba mulai dari menempatkan kendaraan yang belum dimuat, mengubah orientasi, memindahkan kendaraan, hingga menukar posisi kendaraan. Setiap kandidat diperiksa valid tidaknya beserta nilai objectivenya. Hanya kandidat yang valid dan memiliki nilai objective lebih tinggi yang disimpan untuk digunakan.

```
rows + 1):
                    for x in range(ship.dimensions.w -
cols + 1):
                        new_state =
dict(current_state)
                        new_state[v_id] = Placement(x,
y, orient)

                        if is_valid(new_state,
vehicles, ship):
                            new_value =
value(new_state, vehicles)

                            if new_value >
current_value:

better_neighbors.append(new_state)

        else:
            for orient in Orientation:
                new_state = dict(current_state)
                new_state[v_id] = Placement(
                    posisi.x, posisi.y, orient
                )

                if is_valid(new_state, vehicles,
ship):
                    new_value = value(new_state,
```

```
vehicles)

                    if new_value > current_value:

better_neighbors.append(new_state)



            for y in range(ship.dimensions.l):
                for x in range(ship.dimensions.w):
                    new_state = move(current_state,
v_id, x, y)

                    if is_valid(new_state, vehicles,
ship):
                        new_value = value(new_state,
vehicles)

                        if new_value > current_value:

better_neighbors.append(new_state)



    vehicle_ids = list(vehicles.keys())

    for i in range(len(vehicle_ids)):
        for j in range(i + 1, len(vehicle_ids)):
            new_state = swap(
                current_state,
```

<pre>vehicle_ids[i],
vehicle_ids[j]
)
 	
if is_valid(new_state, vehicles, ship):
new_value = value(new_state, vehicles)
 	
if new_value > current_value:
better_neighbors.append(new_state)
 	
return better_neighbors</pre>	

<pre>def stochastic_hill_climbing(vehicles, ship):
start_time = time.perf_counter()
current_state= initial_state(vehicles,ship)
start_state = dict(current_state)
iterations = 0
history = [value(current_state, vehicles)]
 	
while True:
better_neighbors = find_better_neighbors(current_state, vehicles, ship)
 	
if not better_neighbors:
break
 	
current_state = random.choice(better_neighbors)</pre>
	Bagian ini intinya proses utama Stochastic Hill Climbing, Pada setiap iterasi, program mencari neighbor yang memiliki nilai objective lebih tinggi. Jika tidak ditemukan neighbor yang lebih baik, pencarian dihentikan. Jika masih ada, program memilih salah satu kandidat secara acak, memperbarui state saat ini, dan mencatat nilai objective serta jumlah iterasinya. Setelah pencarian selesai, program menghitung nilai objective akhir dan durasi eksekusi, lalu mengembalikan state awal, state akhir, nilai objective akhir, jumlah iterasi, riwayat nilai objective, dan durasi pencarian.

```
        iterations +=1

        current_value = value(current_state, vehicles)
        history.append(current_value)

    final_state = current_state
    final_value = value(final_state, vehicles)
    duration = time.perf_counter() - start_time

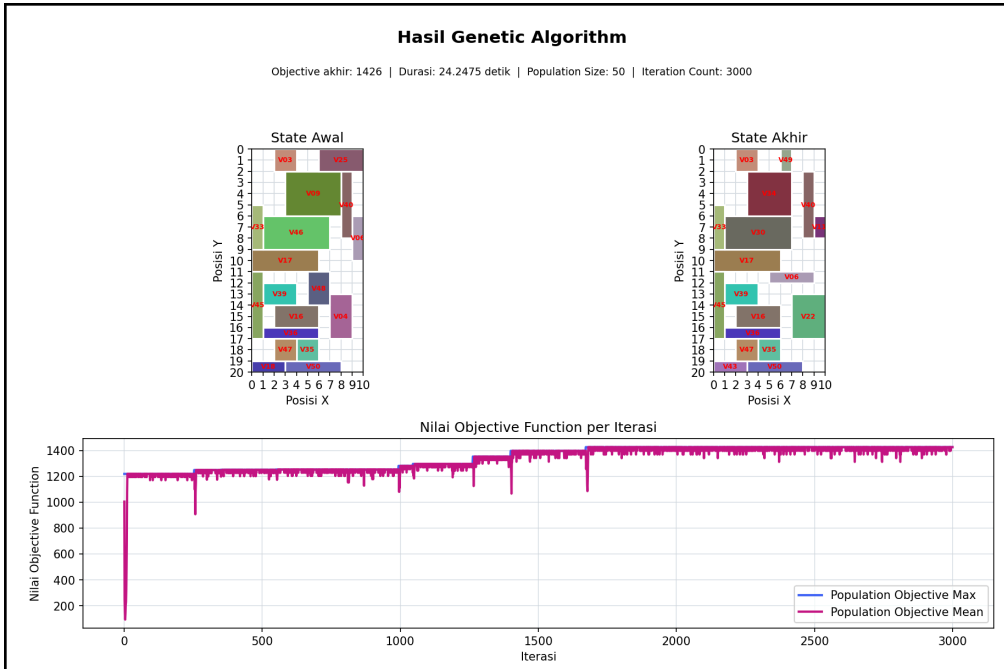
    return{
        "initial_state" : start_state,
        "final_state" : final_state,
        "final_value" : final_value,
        "iterations" : iterations,
        "history" : history,
        "duration" : duration
    }
```

2.3. Hasil Eksperimen tiap Algoritma

2.3.1. Genetic Algorithm

Sesuai spesifikasi, Genetic Algorithm akan dijalankan dengan seed yang berbeda-beda sebanyak total **18 kali, dengan rincian sebagai berikut:**

- **Banyak iterasi sebagai kontrol dan jumlah populasi bervariasi**
  - 3000 iterasi dengan 50 populasi: 3x percobaan
  - 3000 iterasi dengan 100 populasi: 3x percobaan
  - 3000 iterasi dengan 150 populasi: 3x percobaan
- **Jumlah populasi sebagai kontrol dan banyak iterasi bervariasi**
  - 100 populasi dengan 1000 iterasi : 3x percobaan
  - 100 populasi dengan 2500 iterasi : 3x percobaan
  - 100 populasi dengan 5000 iterasi : 3x percobaan



3000 iterasi dengan 50 populasi

Grafik 1:
Objective akhir: 1426
Durasi: 24.2475s

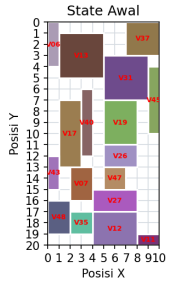
Grafik 2:
Objective akhir: 1432
Durasi: 22.1126s

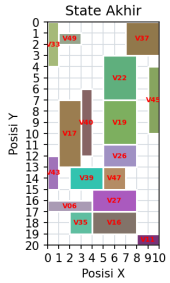
Grafik 3:
Objective akhir: 1482
Durasi: 24.9301s

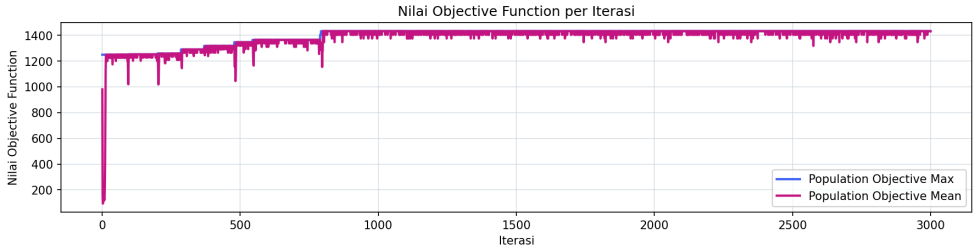
Dengan **ukuran populasi yang relatif kecil**, waktu pencarian berlangsung cukup cepat (**22-25 detik**). Hasil tertinggi diperoleh pada percobaan ketiga sebesar 1482. Pada grafik, terlihat mean populasi dengan cepat mendekati nilai maksimum di awal iterasi. Hal ini menandakan **populasi cepat mengalami konvergensi**, namun berisiko terjebak pada local maximum lebih awal karena terbatasnya keberagaman genetik.

Hasil Genetic Algorithm

Objective akhir: 1432 | Durasi: 22.1126 detik | Population Size: 50 | Iteration Count: 3000

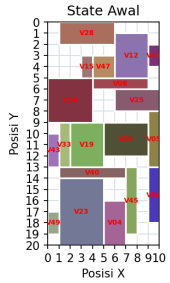


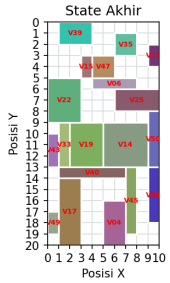


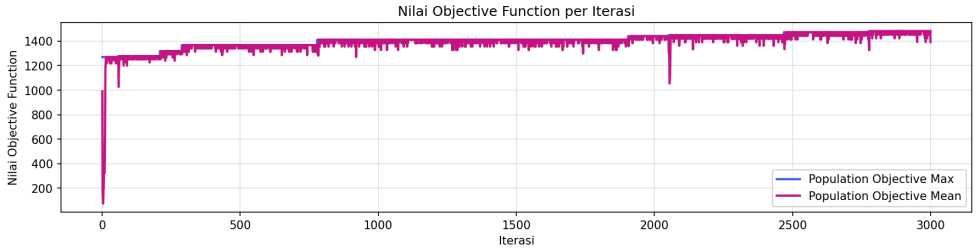


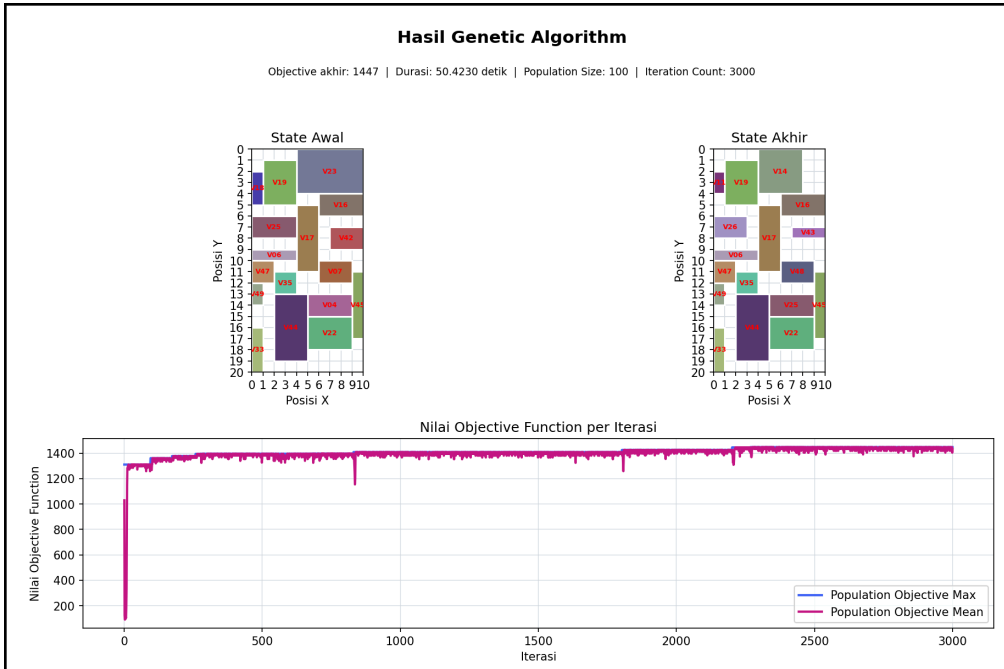
Hasil Genetic Algorithm

Objective akhir: 1482 | Durasi: 24.9301 detik | Population Size: 50 | Iteration Count: 3000









3000 iterasi dengan 100 populasi

Grafik 1:
Objective akhir: 1447
Durasi: 50.4230s

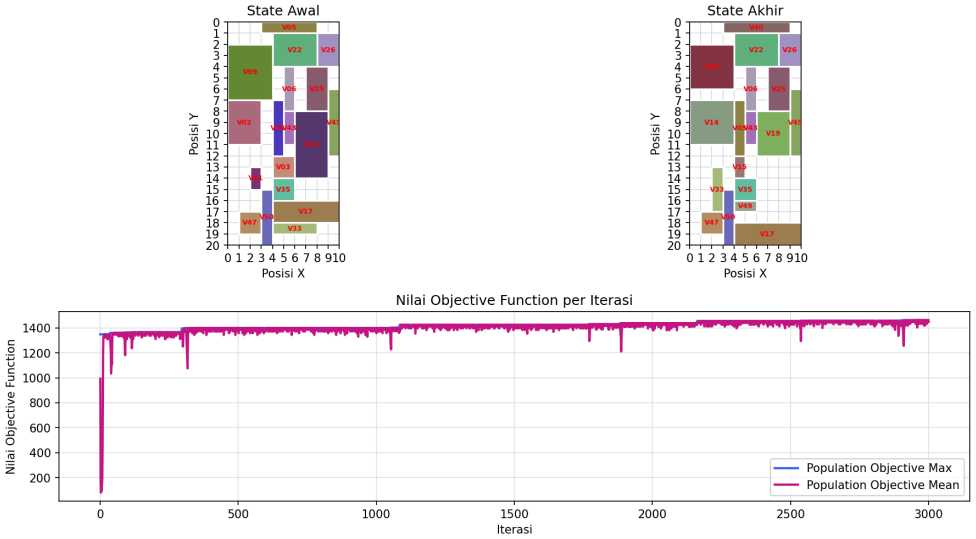
Grafik 2:
Objective akhir: 1463
Durasi: 51.5415s

Grafik 3:
Objective akhir: 1438
Durasi: 51.5531s

Peningkatan jumlah populasi menjadi 100 membuat **durasi eksekusi naik dua kali lipat menjadi 50-51 detik**. Nilai **objective akhir yang diperoleh lebih stabil dan konsisten** berada pada rentang 1438–1463. Keberagaman yang lebih tinggi memungkinkan proses eksplorasi berjalan lebih merata sebelum terjadi konvergensi.

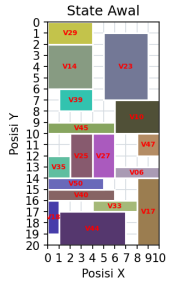
Hasil Genetic Algorithm

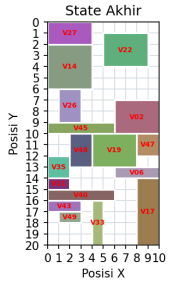
Objective akhir: 1463 | Durasi: 51.5415 detik | Population Size: 100 | Iteration Count: 3000

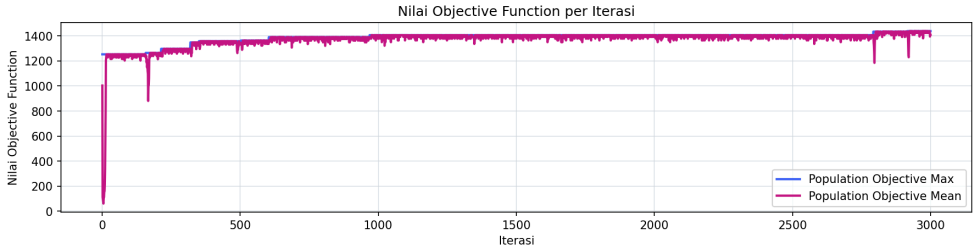


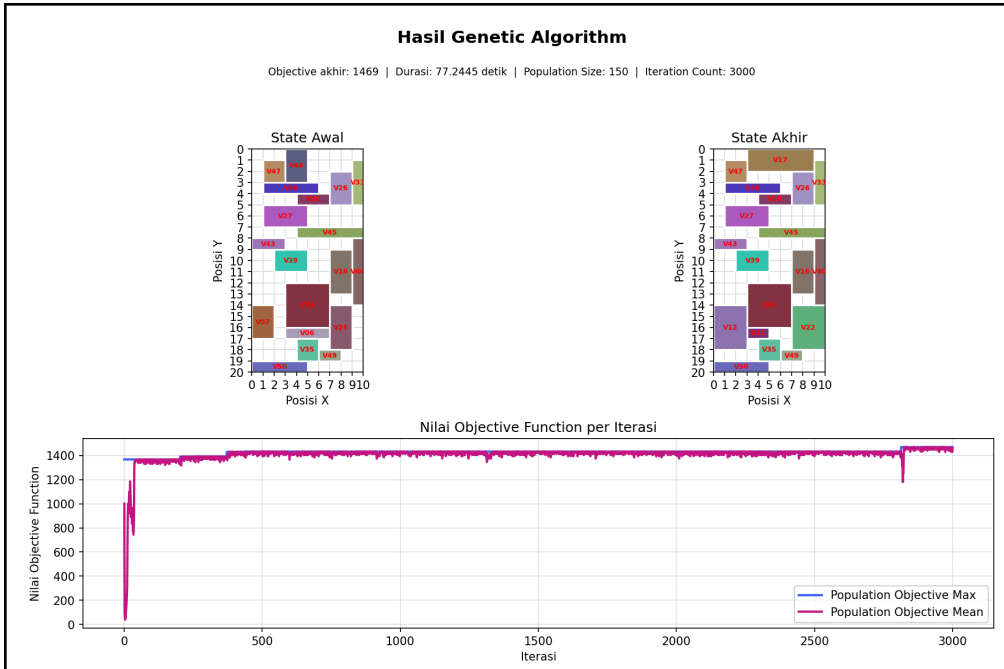
Hasil Genetic Algorithm

Objective akhir: 1438 | Durasi: 51.5531 detik | Population Size: 100 | Iteration Count: 3000









3000 iterasi dengan 150 populasi

Grafik 1:
Objective akhir: 1469
Durasi: 77.2445s

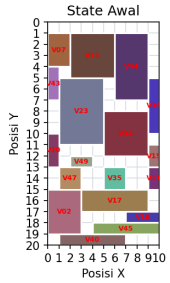
Grafik 2:
Objective akhir: 1401
Durasi: 72.4174s

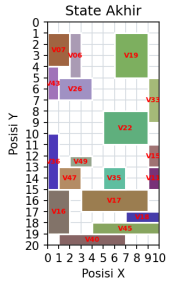
Grafik 3:
Objective akhir: 1463
Durasi: 72.3948s

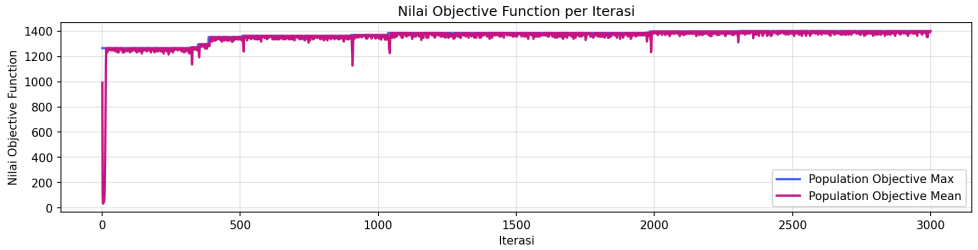
Menggunakan populasi terbesar (150) pada 3000 iterasi membutuhkan waktu komputasi yang paling lama di kelompoknya (**72-77 detik**). Pencarian berhasil mencapai solusi 1469 pada percobaan pertama. Meskipun eksplorasi genetik sangat luas, **terjadi variansi nilai akhir yang cukup jauh pada percobaan kedua** (1401), menunjukkan bahwa **menambah populasi tanpa menambah jumlah iterasi tidak selalu menjamin peningkatan kualitas** solusi secara linier.

Hasil Genetic Algorithm

Objective akhir: 1401 | Durasi: 72.4174 detik | Population Size: 150 | Iteration Count: 3000

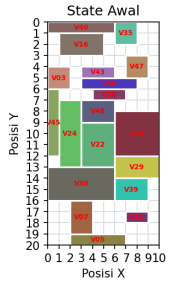


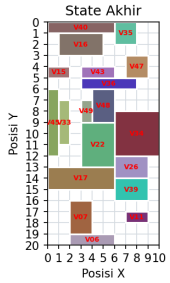


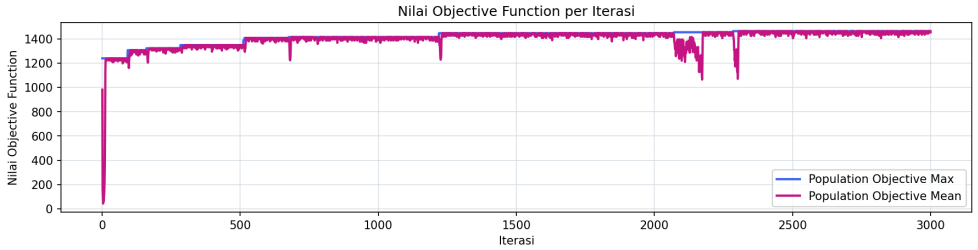


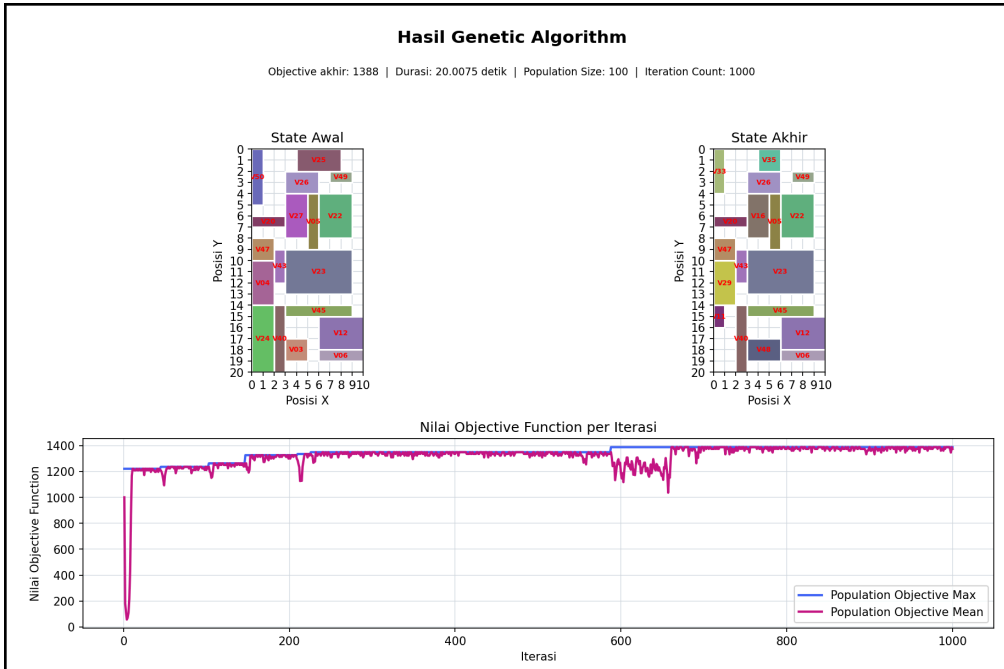
Hasil Genetic Algorithm

Objective akhir: 1463 | Durasi: 72.3948 detik | Population Size: 150 | Iteration Count: 3000









100 populasi dengan 1000 iterasi

Grafik 1:
Objective akhir: 1388
Durasi: 20.0075s

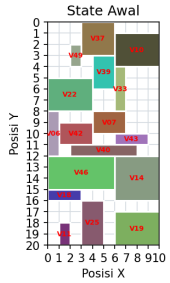
Grafik 2:
Objective akhir: 1349
Durasi: 19.3657s

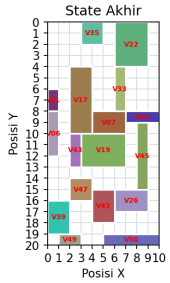
Grafik 3:
Objective akhir: 1342
Durasi: 19.7329s

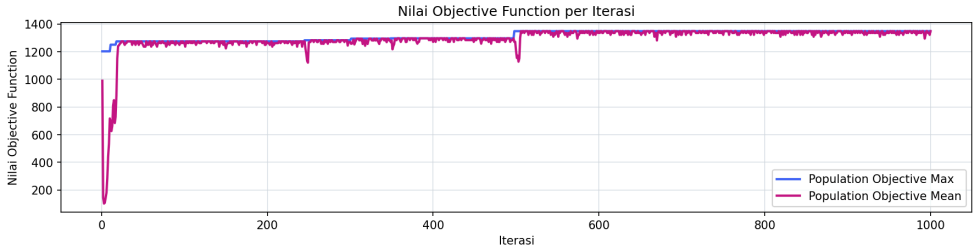
Dengan membatasi iterasi hanya 1000 generasi, **durasi proses pencarian sangat singkat** (19–20 detik). Namun, **nilai *objective* yang dicapai merupakan yang terendah** di antara seluruh skenario (berkisar antara 1342–1388). **Jumlah 1000 iterasi belum cukup memberikan kesempatan** bagi algoritma untuk melakukan eksploitasi dan evolusi genetik hingga mencapai konvergensi optimal.

Hasil Genetic Algorithm

Objective akhir: 1349 | Durasi: 19.3657 detik | Population Size: 100 | Iteration Count: 1000



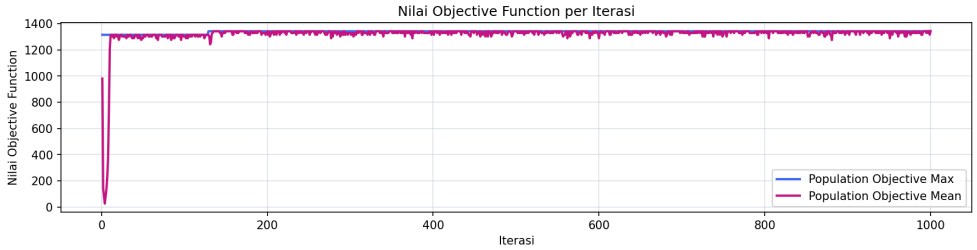


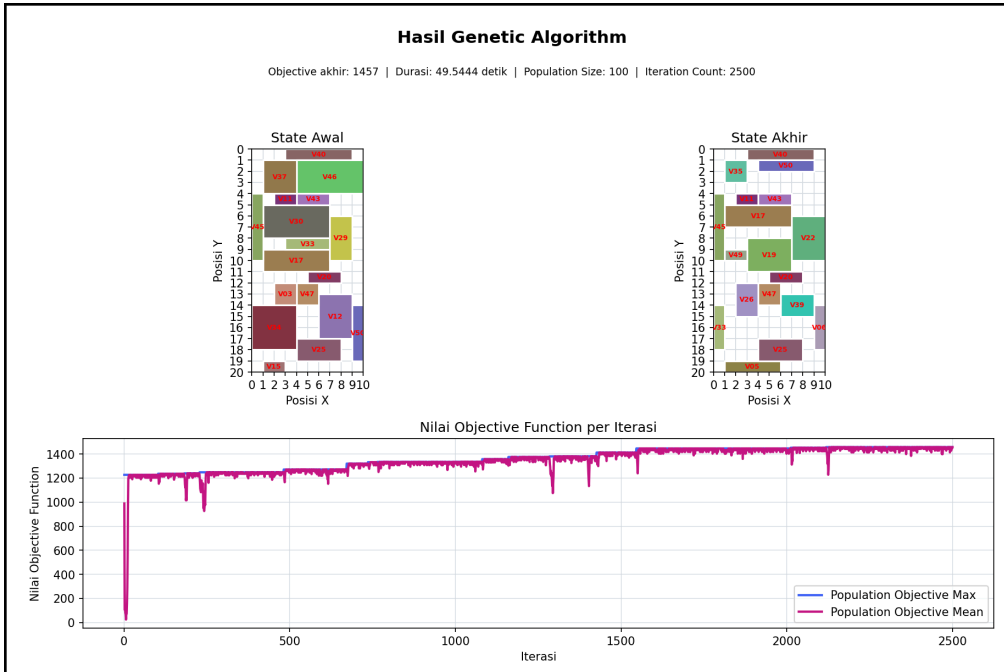


Hasil Genetic Algorithm

Objective akhir: 1342 | Durasi: 19.7329 detik | Population Size: 100 | Iteration Count: 1000







100 populasi dengan 2500 iterasi

Grafik 1:
Objective akhir: 1457
Durasi: 49.5444s

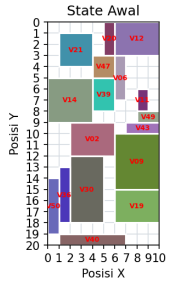
Grafik 2:
Objective akhir: 1436
Durasi: 49.8666s

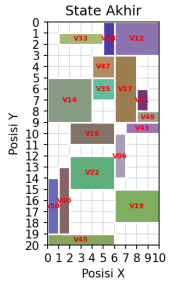
Grafik 3:
Objective akhir: 1458
Durasi: 48.6400s

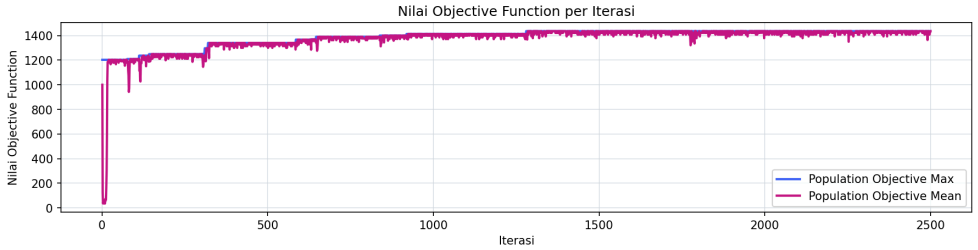
Penambahan jumlah iterasi menjadi 2500 pada populasi 100 **memberikan peningkatan kualitas solusi yang signifikan ke kisaran 1436-1458** dengan **durasi eksekusi 48-49 detik**, jauh lebih baik dan stabil dibanding eksperimen sebelumnya

Hasil Genetic Algorithm

Objective akhir: 1436 | Durasi: 49.8666 detik | Population Size: 100 | Iteration Count: 2500

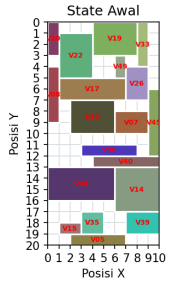


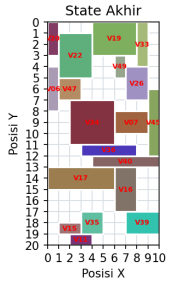


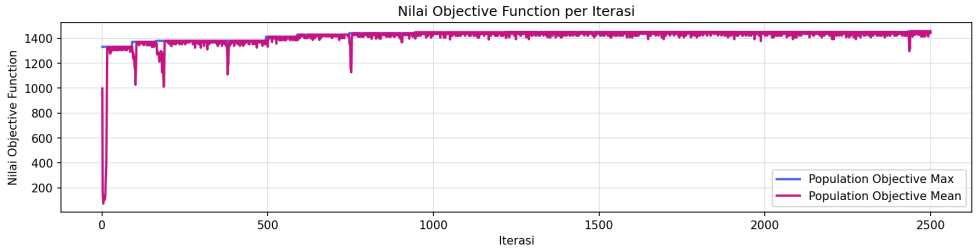


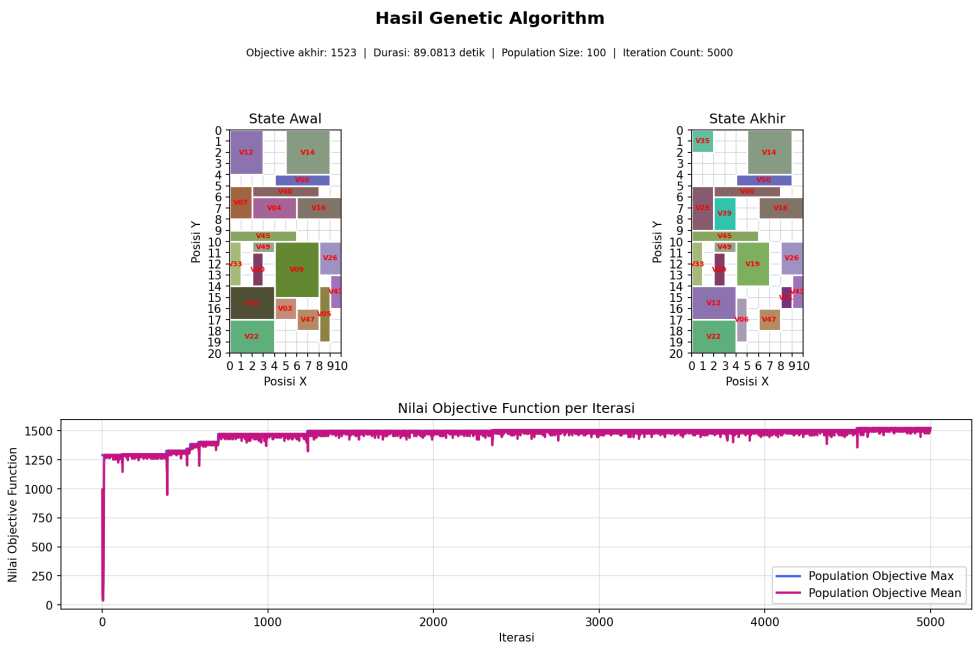
Hasil Genetic Algorithm

Objective akhir: 1458 | Durasi: 48.6400 detik | Population Size: 100 | Iteration Count: 2500









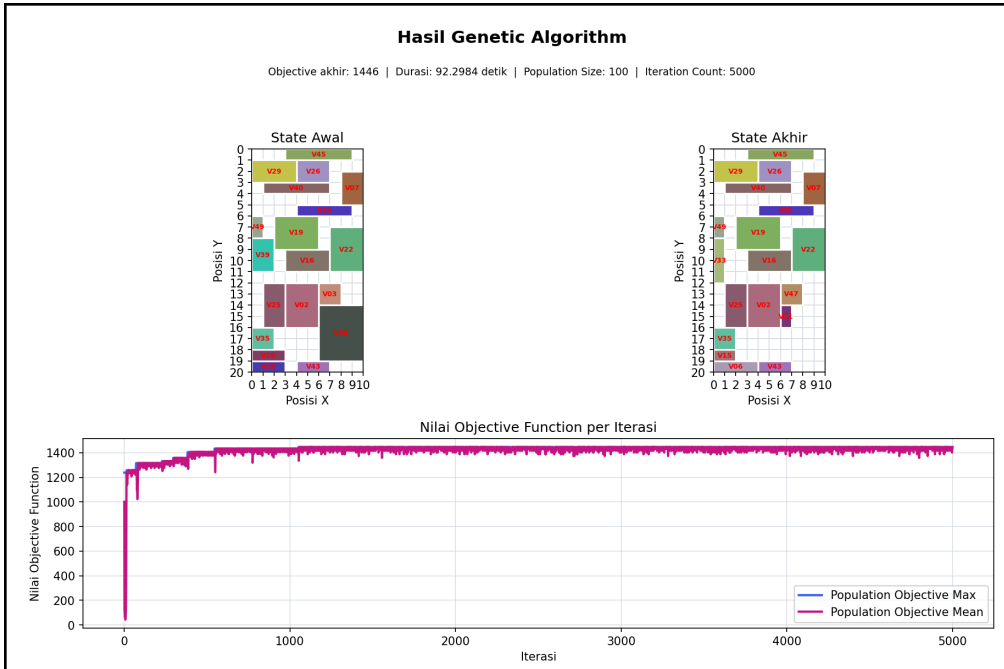
100 populasi dengan 5000 iterasi

Grafik 1:
Objective akhir: 1523
Durasi: 89.0813s

Grafik 2:
Objective akhir: 1446
Durasi: 92.2984s

Grafik 3:
Objective akhir: 1515
Durasi: 93.4303s

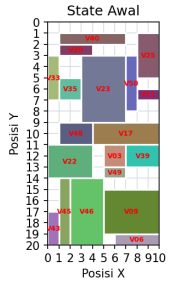
Konfigurasi dengan iterasi terbanyak (5000 iterasi) **menghasilkan nilai objective tertinggi** di seluruh eksperimen Genetic Algorithm, yaitu mencapai **1523** pada percobaan pertama dan 1515 pada percobaan ketiga. **Waktu pencariannya mencapai 89-93 detik**. Kombinasi populasi yang memadai dan jumlah generasi yang panjang memberikan waktu yang cukup bagi mekanisme crossover dan mutasi untuk menemukan susunan kargo yang mendekati global maximum, walau

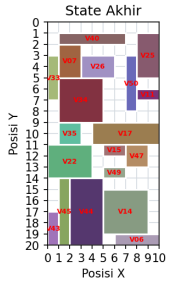


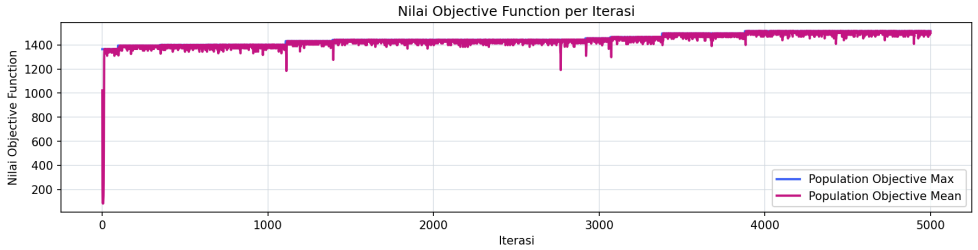
memang belum bisa seefisien dan sedekat algoritma lain.

Hasil Genetic Algorithm

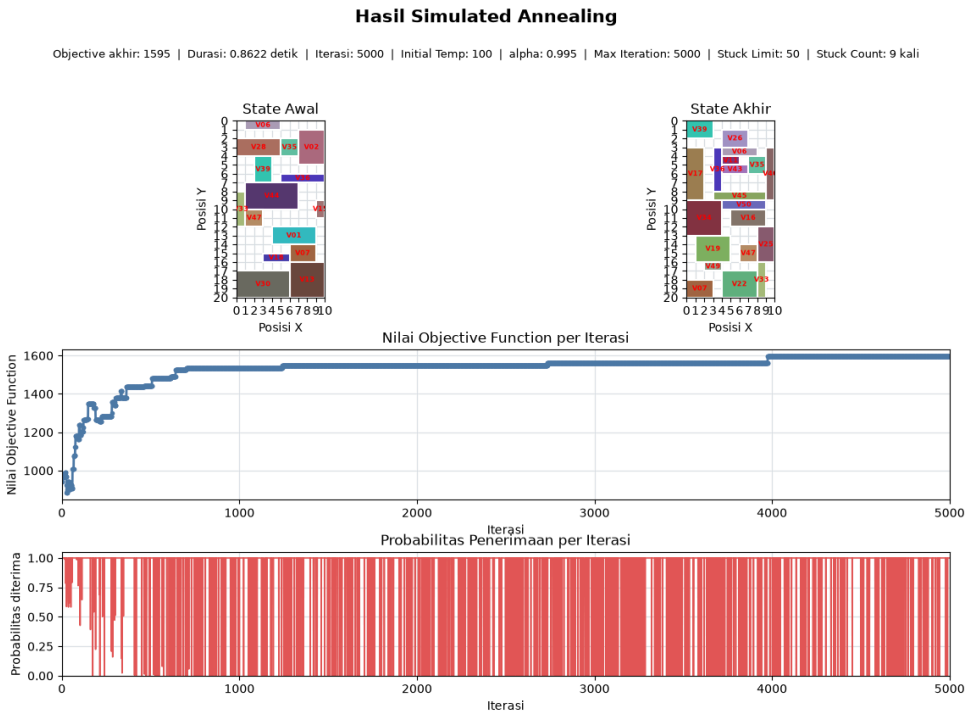
Objective akhir: 1515 | Durasi: 93.4303 detik | Population Size: 100 | Iteration Count: 5000







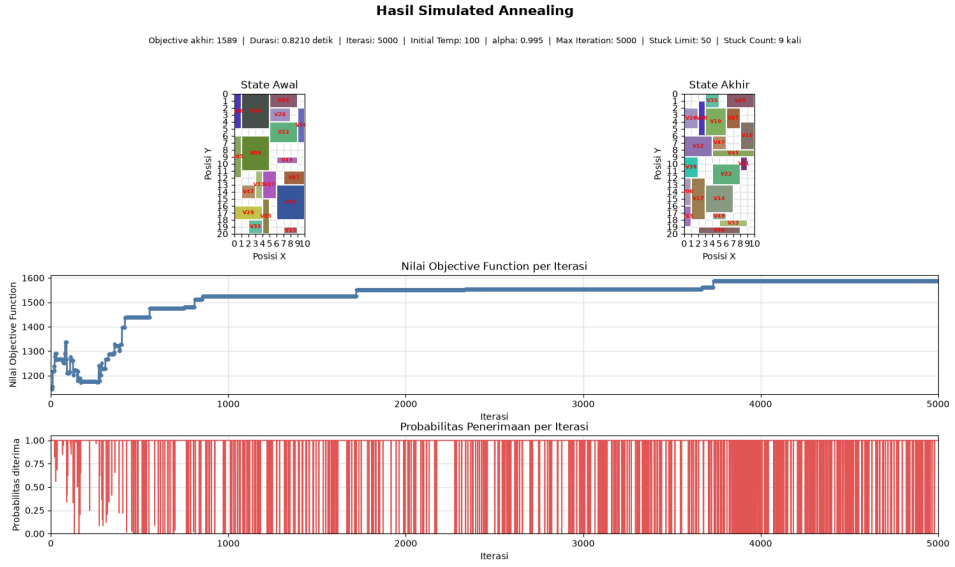
2.3.2. Simulated Annealing

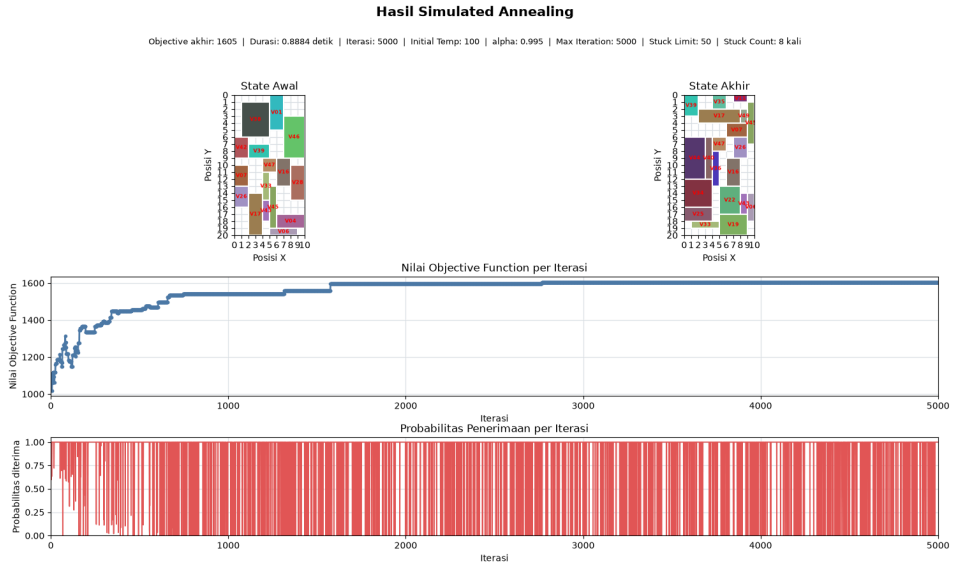


Simulated annealing dijalankan sebanyak 3 kali dengan seed yang berbeda.

Grafik pertama menunjukkan nilai objective function pada setiap iterasi. Pada tiga percobaan, nilai objective naik dari sekitar 1000 menjadi 1595, 1605, dan 1589, sehingga hasil akhir antar percobaan cukup konsisten. Di awal pencarian, kurva naik turun sebelum akhirnya naik secara stabil. Setelah sekitar 1000 iterasi , kurva mulai stabil dan hanya naik sedikit sesekali, misalnya di sekitar iterasi 1600, 2700, dan 3900.

Grafik kedua menunjukkan probabilitas penerimaan candidate pada setiap iterasi, yaitu p = e^(delta/T) untuk candidate yang lebih buruk dan p = 1 untuk candidate yang sama atau lebih baik. Grafik ini menjelaskan alasan mengapa grafik objective berbentuk seperti itu. Pada awal



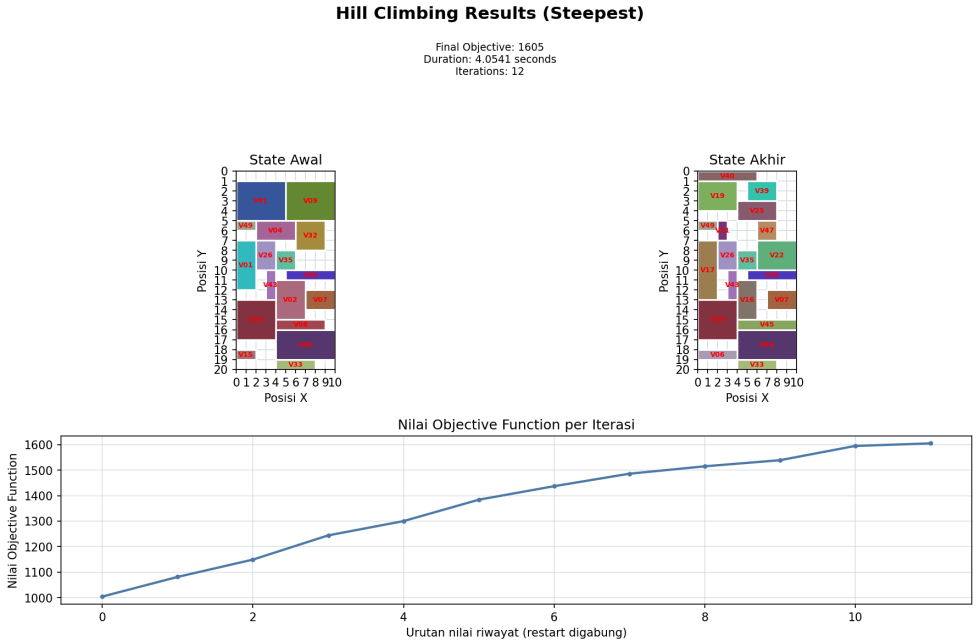


pencarian, temperatur masih tinggi, sehingga probabilitas masih bervariasi diantara 0 dan 1. Solusi yang lebih buruk masih punya peluang diterima. Inilah kenapa grafik objective sempat turun di awal. Hal ini memungkinkan algoritma menemukan state terbaik tanpa stuck di local maximum.

Dengan alpha 0,995, T sisa sekitar 0,7 pada iterasi 1000. Peluang menerima solusi yang lebih buruk mengecil. Karena itu algoritma pasti tidak akan memindahkan ke state yang lebih buruk karena temperatur sudah mendekati dan yang tersisa hanya 1 (candidate sama atau lebih baik, selalu diterima). Saat T mendekati 0, simulated Annealing berperilaku seperti hill climbing biasa, sehingga kurva objective mendatar dan hanya berubah ketika ditemukan candidate yang lebih baik.

2.3.3. Hill-Climbing

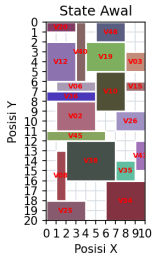
a. Hill-Climbing Steepest Ascent

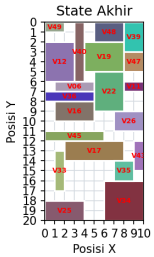


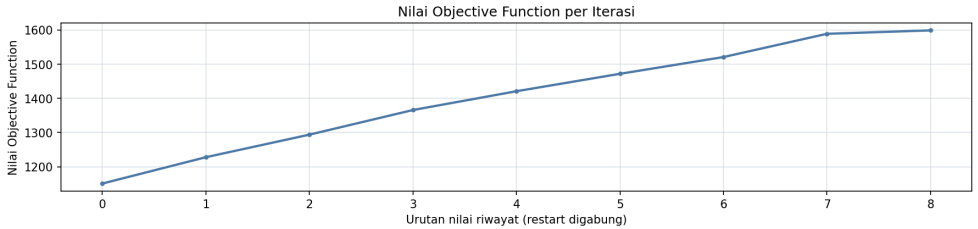
Objective tertinggi yang didapat adalah 1605 pada percobaan pertama. Percobaan kedua sebenarnya punya hasil yang tidak jauh beda, yaitu 1599, bahkan waktu eksekusinya lebih cepat, dari 4,0541 detik menjadi 2,8849 detik. Ketika percobaan ketiga, itu mendapatkan objective 1527. Perbedaan ini bisa terjadi karena proses algoritmanya memilih tetangga dengan peningkatan objective terbaik di setiap langkahnya. Tetapi, pilihan terbaik dari belum tentu menghasilkan hasil terbaik secara keseluruhan. Jadi, meskipun proses pencarian di percobaan kedua lebih cepat, hasil akhirnya masih sedikit dibandingkan percobaan pertama.

Hill Climbing Results (Steepest)

Final Objective: 1599
Duration: 2.8849 seconds
Iterations: 9



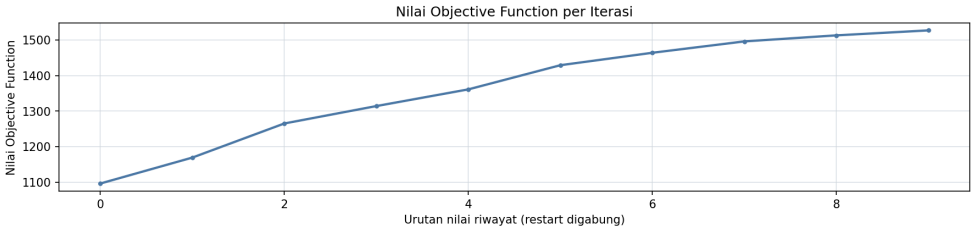




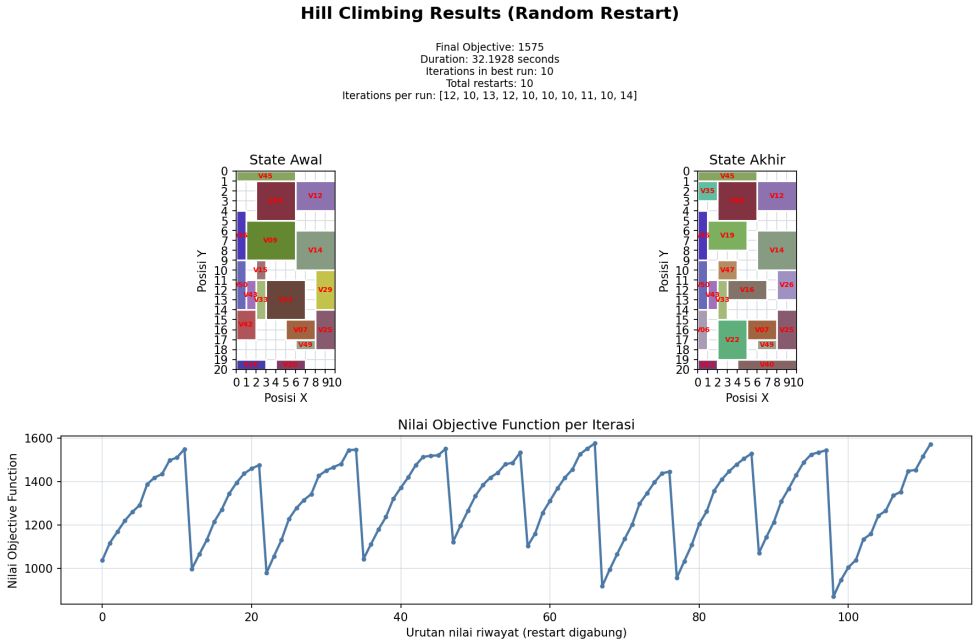
Hill Climbing Results (Steepest)

Final Objective: 1527
Duration: 3.4157 seconds
Iterations: 10

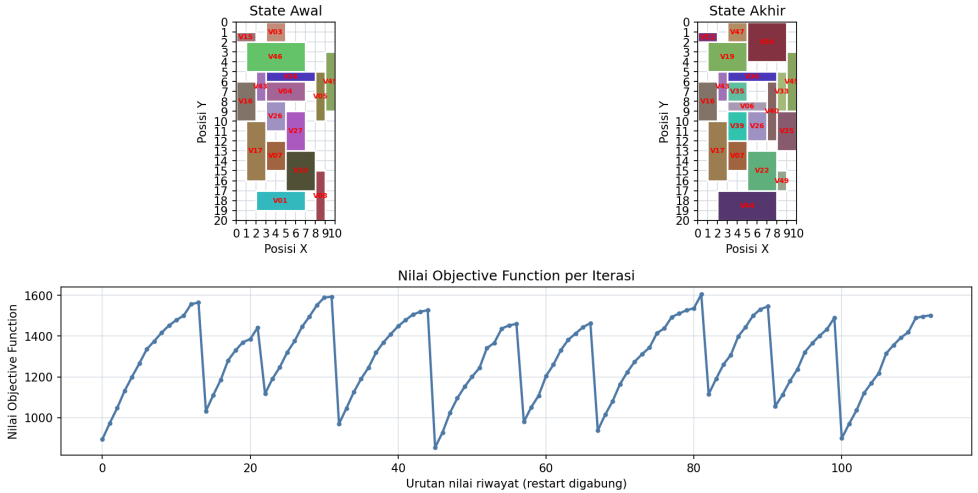




b. Random Restart Hill-Climbing



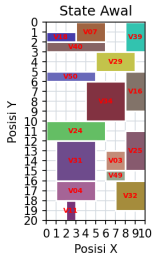
Percobaan kedua mendapatkan hasil paling bagus dengan objective 1605 dan waktu eksekusi sekitar 31 detikan. Percobaan ketiga juga mendapatkan hasil yang lumayan dekat, yaitu 1600, tetapi membutuhkan waktu sekitar 35 deitk. Sedangkan untuk percobaan pertama objectivenya 1575 dalam waktu sekitar 32 detik. Ketiganya sama-sama melakukan 10 kali restart, tetapi hasil akhirnya tetap berbeda. Ini karena setiap restart memulai pencarian dari state awal yang berbeda, jadi jalur pencariannya juga bisa berbeda. Restart memang memberi berusaha untuk mencari solusi lain ketika pencarian sebelumnya berhenti di lokal maksima atau ketika sudah tidak ada nilai yang lebih baik, tetapi bukan berarti setiap restart pasti menghasilkan solusi yang lebih baik. Dalam konteks ini, percobaan kedua kebetulan

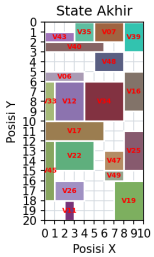


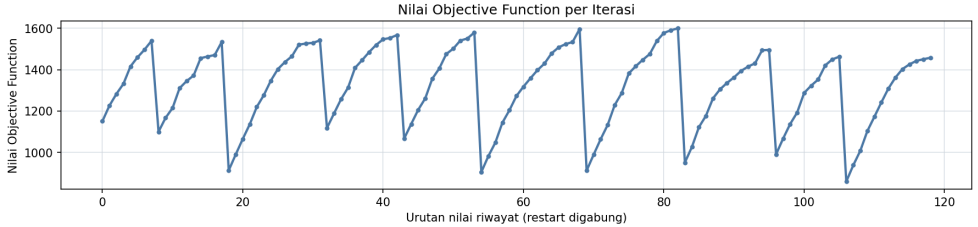
mendapatkan hasil tertinggi dengan waktu paling singkat.

Hill Climbing Results (Random Restart)

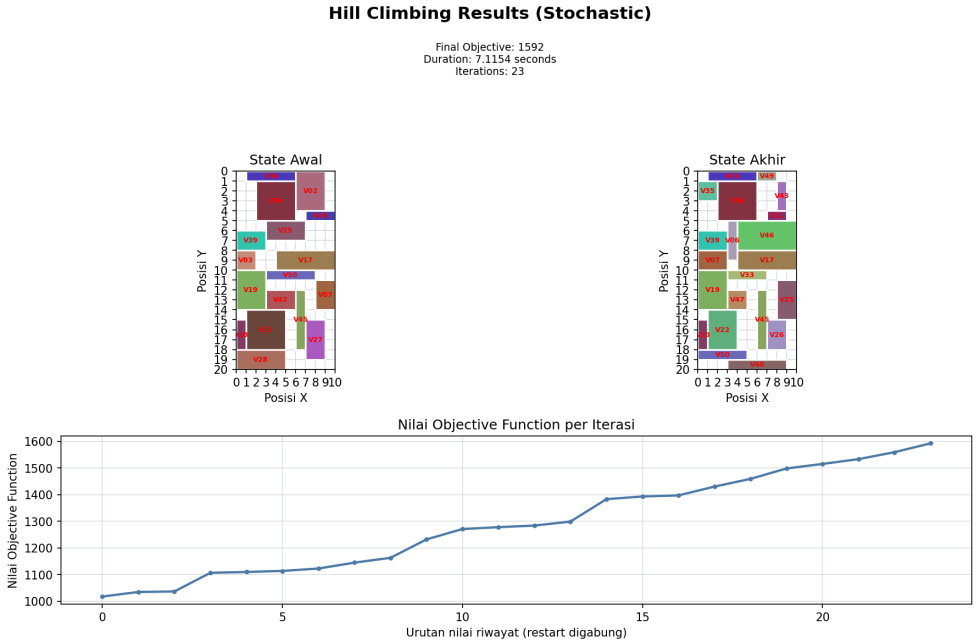
Final Objective: 1600
Duration: 35.6700 seconds
Iterations in best run: 14
Total restarts: 10
Iterations per run: [8, 10, 14, 11, 11, 15, 14, 13, 10, 13]







c. Stochastic Hill-Climbing

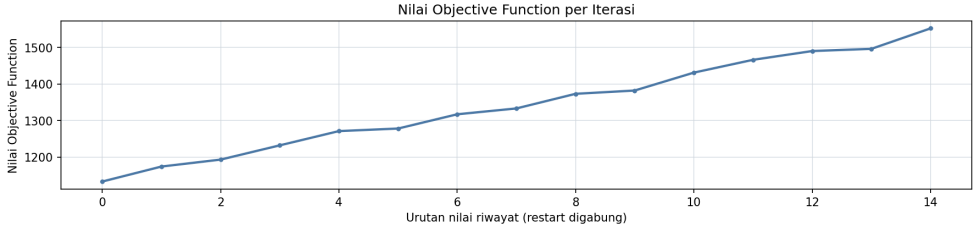


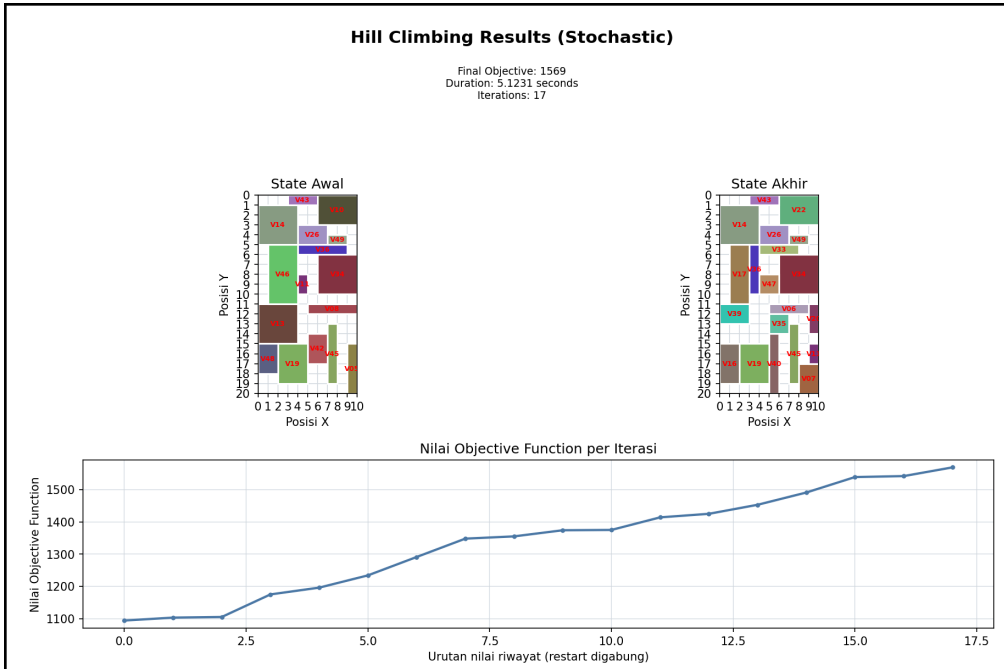
Hasilnya kalau dilihat itu cukup berbeda di setiap percobaan. Objective tertinggi didapat pada percobaan pertama, yaitu 1592, dengan waktu sekitar 7 detik dalam 23 iterasi. Untuk percobaan kedua, itu hanya membutuhkan 14 iterasi dengan waktu sekitar 4 detik, tetapi objectivenya juga paling rendah, yaitu 1552. Percobaan ketiga berada di antara keduanya dengan objective 1569. Cara kerja Stochastic Hill Climbing adalah memilih tetangga secara acak dari kandidat yang memenuhi kriteria. Jadi, kandidat yang dipilih belum tentu yang punya peningkatan objective paling tinggi. Nah, karena pilihan tetangganya bisa berbeda, hasil pencarian di setiap percobaan juga bisa berbeda. Pada percobaan pertama, proses pencariannya memang lebih lama, tetapi dari susunan kendaraannya menghasilkan objective lebih tinggi.

Hill Climbing Results (Stochastic)

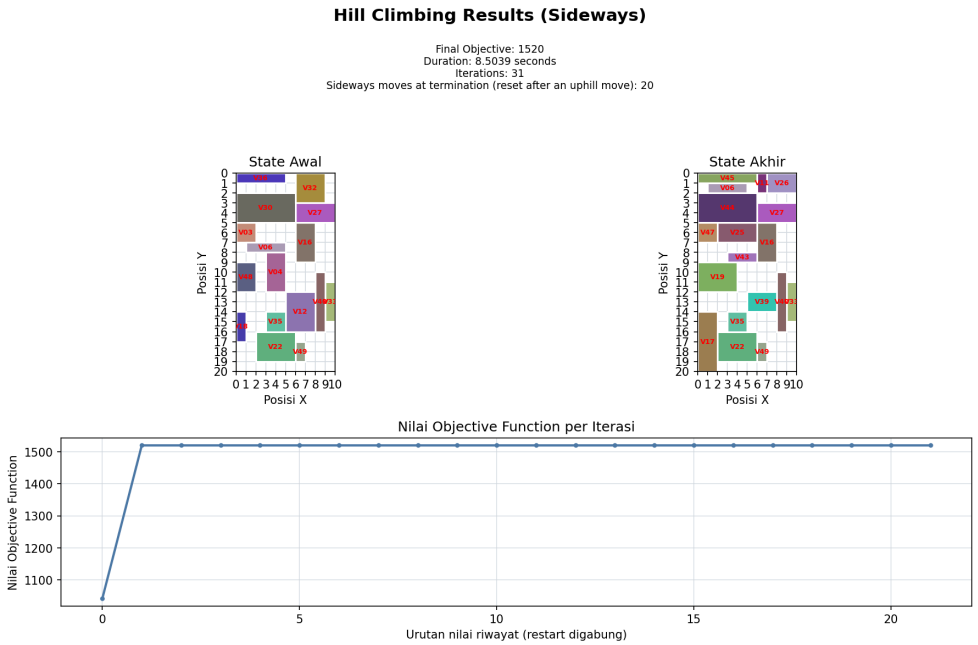
Final Objective: 1552
Duration: 4.0537 seconds
Iterations: 14



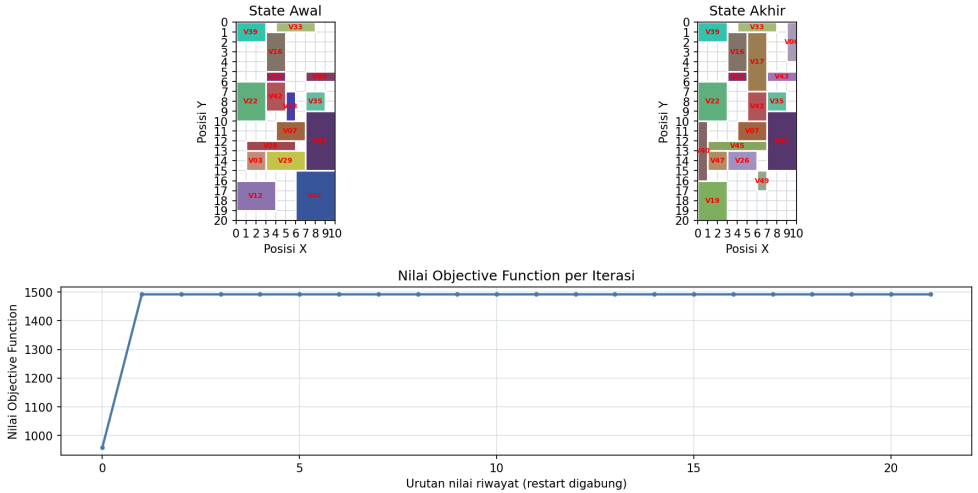



Hill Climbing Results (Stochastic)
Final Objective: 1569
Duration: 5.1231 seconds
Iterations: 17
State Awal
State Akhir
Posisi X
Posisi Y
Nilai Objective Function per Iterasi
Nilai Objective Function
Urutan nilai riwayat (restart digabung)

d. Hill-Climbing with Sideways Moves



Pada percobaan ini, objective tertinggi didapat pada percobaan ketiga, yaitu 1548. Percobaan pertama menghasilkan 1520, sedangkan percobaan kedua hanya 1492. Walaupun hasilnya berbeda, ketiganya sama-sama berjalan selama 31 iterasi dan berhenti setelah mencapai batas 20 langkah sideways. Jika dilihat dari grafik, objective meningkat di awal pencarian, lalu nilainya cenderung tetap sampai algoritma berhenti. Ini terjadi karena sideways move memungkinkan algoritma berpindah ke state lain yang punya objective sama sehingga pencarian masih bisa dilanjutkan meskipun belum ada peningkatan nilai. Namun, langkah ini tetap punya batas. Semisal batas sideways sudah tercapai, algoritma akan berhenti meskipun belum menemukan objective yang lebih tinggi. Oleh karena itu, /walaupun jumlah iterasi ketiganya sama, objective

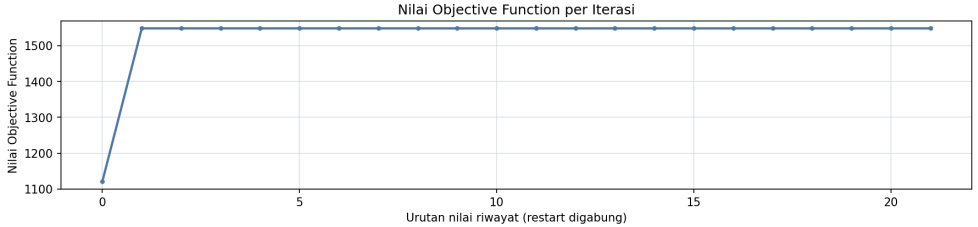


akhirnya tetap berbeda.

Hill Climbing Results (Sideways)

Final Objective: 1548
Duration: 8.6827 seconds
Iterations: 31
Sideways moves at termination (reset after an uphill move): 20





BAB III
KESIMPULAN

3.1. Kesimpulan

Dari hasil percobaan yang kami lakukan, algoritma SA berhasil mendapatkan objective tertinggi dengan waktu eksekusi paling singkat. Algoritma Steepest Ascent juga memperoleh objective yang sama dalam waktu lebih cepat, sedangkan Random Restart membutuhkan waktu lebih lama karena melakukan pencarian berulang dari beberapa state awal. Sementara itu, Stochastic Hill-Climbing dan Hill-Climbing with Sideways Moves menghasilkan objective yang lebih rendah karena hasil pencariannya dipengaruhi oleh pemilihan tetangga dan batas langkah yang digunakan.

Genetic Algorithm membutuhkan waktu lebih lama, tetapi objective yang diperoleh masih lebih rendah dibandingkan beberapa algoritma lainnya. Hal ini bisa dipengaruhi oleh proses crossover yang belum sepenuhnya mempertimbangkan batasan penempatan kendaraan di kapal. Dengan demikian, perbedaan hasil percobaan juga dipengaruhi oleh cara kerja dan penerapan masing-masing algoritma

3.2. Keterbatasan

Pengujian ini masih terbatas pada jumlah percobaan dan kondisi penggunaan algoritma sehingga hasilnya belum sepenuhnya menggambarkan performa setiap algoritma dalam kondisi yang berbeda.

3.3. Saran

Untuk pengujian selanjutnya, jumlah percobaan dapat ditambah agar hasil perbandingan lebih meyakinkan. Selain itu, setiap algoritma dapat dicoba dengan beberapa pengaturan yang berbeda untuk melihat apakah hasil yang diperoleh bisa lebih baik dan waktu pencariannya lebih singkat.

BAB IV

PEMBAGIAN TUGAS

NIM	Nama	Bagian
18224077	Naufalziyadh Alif Bintang S	Kerangka repo (struktur folder MVC, .gitignore, main.py). entities.py (representasi data, effective_size, occupied_cells). neighbor.py (rotate, move, swap). initial.py (initial_state). simulated_annealing.py (Simulated Annealing), generate.py (data generator).
18224099	M Afif Habiburrahman	Mengimplementasikan dan ideasi genetic.py, main.py, Folder view

18224111	Raditya Nanda Dhevara	Implementasi validation.py, objective.py, Steepest Ascent (hc_steepest.py), Sideways Move (hc_sideways.py), dan Random Restart Hill Climbing (hc_random.py).

BAB V

REFERENSI

https://www.knowledgegate.ai/blog/local-search-fundamentals-explained

https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_parent_selection.htm

https://www.knowledgegate.ai/blog/local-search-fundamentals-explained

https://www.geeksforgeeks.org/dsa/genetic-algorithms/